POSTS AND TELECOMMINICATIONS INSTITUTE OF TECHNOLOGY

Department of Information Technology 1

ANALYSIS AND DESIGN OF INFORMATION SYSTEM

Subject No. : **21**
Student : **Nguyễn Bình Minh**
Student's ID : **B22DCDT192**
Class : **E22HTTT**

Hà Nội – 2025

1. **The glossary list**

No	Key words	Meaning
Key words related to human		
1	Management Staff	Staff responsible for managing dish information, adding menu combo, and viewing statistics about dishes, ingredients, customers, and suppliers.
2	Warehouse Staff	Staff responsible for importing materials from suppliers, managing supplier information.
3	Sale staff	Staff responsible for receiving customers, taking orders, processing payments at the table, creating membership cards, confirming reservations, and handling online orders
4	Customer	An individual who interacts with a restaurant online to search, book a table and purchase food
5	Supplier	Partner providing ingredients to the restaurant
Key words related to object		
6	Dish	A prepared food item offered on the restaurant's menu. Each dish may include a name, description, price, and list of required ingredients. Dishes are the primary products sold to customers.
7	Ingredients	The raw food components used to create dishes. Examples include vegetables, meat, spices, or sauces. Managing ingredients involves monitoring stock levels, freshness, supplier details, and ensuring enough supply to meet customer demand.

8	Dish information	Detailed data related to each dish, including recipe instructions, nutritional values, preparation time, price, and linked ingredients. This information helps chefs prepare the dish consistently and enables staff to provide accurate details to customers.
9	Materials	All consumables and non-consumables used in the restaurant, not only food ingredients but also packaging, napkins, or kitchen supplies. Materials management ensures the restaurant runs smoothly and avoids shortages.
10	Orders	Requests placed by customers. Orders can be made in person at the restaurant, through table service, or online platforms. Each order typically includes order ID, customer details, ordered items, quantity, price, and order status.
11	Payments	Financial transactions made by customers to settle their orders. Payments may occur at the cashier, directly at the table, or online. The system records payment methods (cash, credit card, digital wallet), the total amount, and the time of transaction
12	Table	A piece of furniture with a flat top and one or more legs, providing a level surface on which objects may be placed, and that used for seatings and food placing for the customer in this scenario
13	Membership card	A loyalty program card issued to frequent customers, containing customer ID, name, contact details, and accumulated points or benefits. Membership cards encourage repeat visits by offering discounts, promotions, or exclusive offers.
14	Table reservation	An advance arrangement between a customer and a restaurant to secure a specific table for a

		specific time and date, guaranteeing a designated spot upon arrival and avoiding the need to wait in line
15	Online	Represents the digital services provided by the restaurant, such as online ordering, reservations, or payment systems. Online platforms improve convenience for customers and allow restaurants to expand their reach beyond physical service.
16	Order online	A feature that enables customers to browse the menu, select dishes, and place an order through the internet (via website or mobile app). Online orders may be for delivery, takeaway, or dine-in pre-orders.
17	Customer information	Personal and transactional data about customers, including name, phone number, email, order history, preferences, and loyalty membership details. This information supports customer relationship management and personalized service.
18	Supplier information	Details about suppliers who provide the restaurant. Including supplier names, addresses, contacts, and payment history
19	Table reservation information	Details about booked tables.Including customer name, reservation time, number of people, and confirmation status, so staff can prepare in advance.
20	Invoice	A formal billing document generated for a customer's order. It includes invoice number, customer name, ordered items, quantity, price, tax, discounts, and total payment. Invoices are essential for both financial records and customer transparency.
21	Quantity	The numerical amount of ingredients,

		materials, or ordered dishes. Quantity tracking ensures accurate preparation of dishes, proper billing, and effective inventory control.
22	Statistic	Analytical data summarizing restaurant operations, such as total sales, best-selling dishes, average customer spend, or supplier performance. Statistics help managers make data-driven decisions to improve efficiency and profitability.
23	Combo menus	A set of multiple dishes offered together as a package at a special price. They combine several individual dishes (appetizer + main course + drink) into one package, making it convenient for customers and often more economical.
Key words related to activity		
24	Manage dish information	The activity performed by management staff to add, update, or delete dish details in the system. This includes maintaining dish names, ingredients, categories, prices, and availability to ensure that the restaurant's menu is accurate and up to date
25	Make combo menus	The activity of creating special menu combinations from existing dishes. Management staff can design promotional sets or bundles, define prices, and update them in the system for customer ordering
26	View statistic	The activity of accessing and analyzing statistical reports generated by the system. Management staff can view sales data, popular dishes, supplier activity, or customer trends to support decision-making and performance evaluation
27	Manage supplier	The activity of maintaining records of

	information	suppliers. Warehouse staff can add new suppliers, update contact details, or remove inactive suppliers to ensure accurate supplier management in the system
28	Import materials from suppliers	The activity of recording the purchase and receipt of materials from suppliers. Warehouse staff search suppliers, select required ingredients, specify quantities, confirm transactions, and generate invoices for payment
29	Receive customer	The activity performed by sales staff to welcome customers when they arrive at the restaurant. This may include guiding customers to tables, explaining menu options, and initiating the ordering process
30	Receive payments at the table	The activity of processing customer payments directly at their dining table. Sales staff calculate the total bill, handle cash or card payments, issue receipts, and update the system with the completed transaction
31	Make membership card for customer	The activity of registering and issuing membership cards to customers. Sales staff collect customer details, enter them into the system, and generate a card that allows customers to receive benefits such as discounts or loyalty points
32	Order food online	The activity performed by customers to browse the menu and place food orders through the restaurant's online system. Customers select dishes, specify delivery/pickup options, confirm orders, and make payments digitally
33	Search a table	The activity of looking for available tables in the restaurant system. Customers can check table availability by date, time, and seating capacity before making a reservation

34	Book a table	The activity of reserving a table in advance. Customers choose their preferred time and date, confirm the booking, and the system records the reservation to prevent double-booking

2. Project information

a. Object

- Develop a restaurant management system that allows management staff, warehouse staff, sales staff, and customers to interact through the system.
- Automate processes such as dish management, ingredient imports, customer management, table booking, payments, and supplier management.
- Reduce manual errors, save time, improve transparency, and enhance customer experience.

b. Scope

- Restaurant staff:
  - Log in
- Management staff:
  - View statistic
  - Manage dish information
  - Make combo menus
- Warehouse staff:
  - Import materials from suppliers
  - Manage supplier’s information
- Sale staff:
  - Receive customers
  - Take orders
  - Receive payments
  - Make membership cards
  - Confirm table reservation
  - Confirm online order
- Customer:
  - Search a table
  - Book a table

        - Order foods online

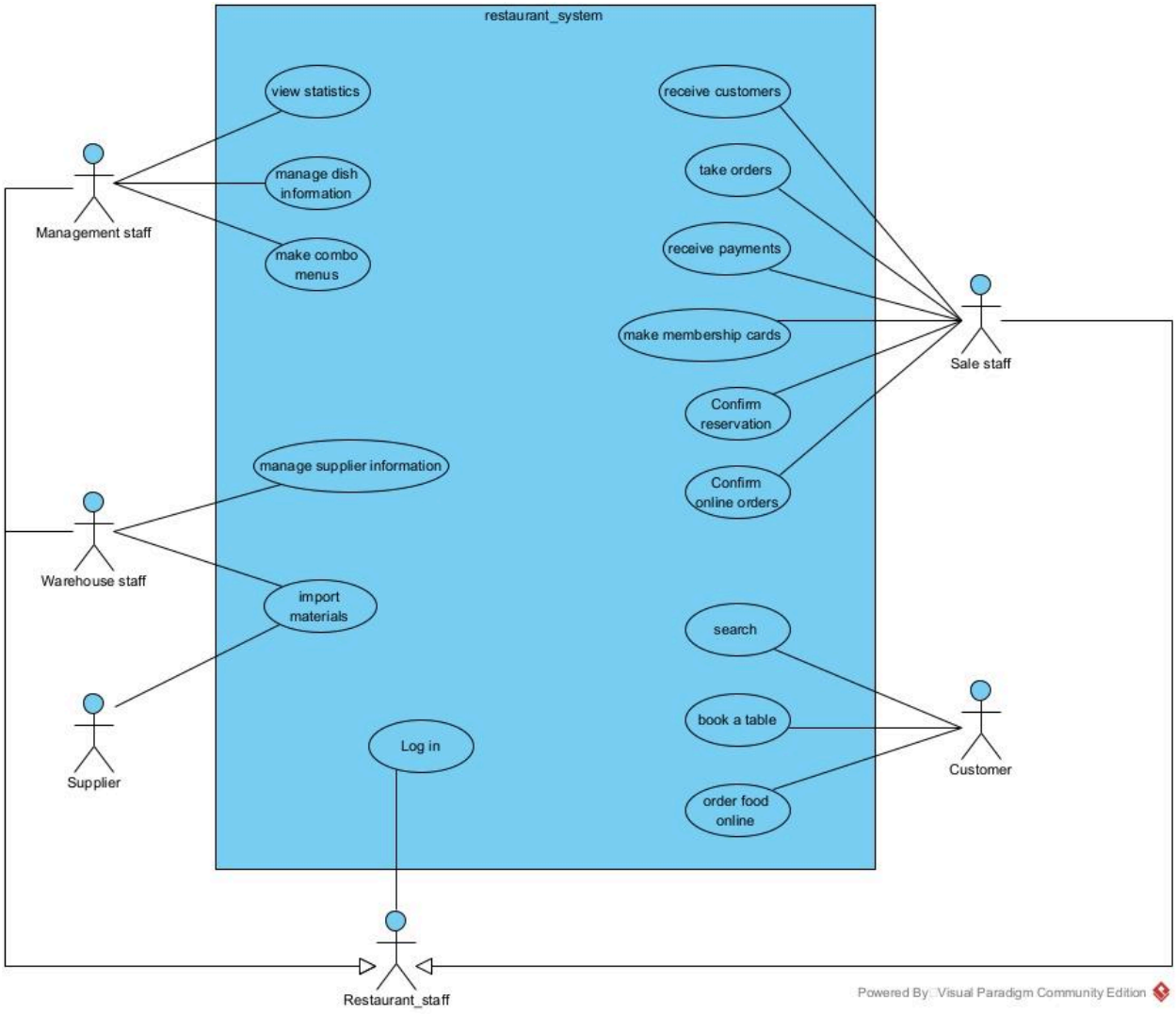
c. How the module works
    - Add new dish (Management staff):
        - Select the menu to manage dishes
        - Select “add dish” function
        - Enter dish information and click save
        - The system stores the information in the database
    - Import ingredients (Warehouse staff):
        - Select the menu to import ingredients
        - Find the supplier by name (add new if not existing)
        - Search for ingredients (add new if not existing)
        - Select ingredients, enter quantity
        - Repeat until the list of ingredients is complete
        - Print the import invoice and confirm payment to the supplier

d. Information about objects
    - Staff: username, password, name, mobile number, email, role
    - Dish: name, category, price, required ingredients.
    - Ingredients: name, unit, stock quantity, supplier.
    - Supplier: name, address, phone.
    - Customer: full name, phone, email, membership card.
    - Order: customer, list of dishes, total price, status.
    - Table reservation: customer, time, number of people.
    - Invoice: content, amount, date.

e. Relationships among objects
    - Customer-Membership: 1-1
    - Customer-Reservation: 1-n
    - Customer-Invoice: 1-n
    - Reservation-Order: 1-n
    - Reservation-Table: n-1
    - Reservation-Invoice: 1-n
    - Order-Invoice: n-1
    - Order-Dish: n-n
    - Order-Combo: n-n
    - Dish-Combo: n-n

3. Use case diagram

a. General use case

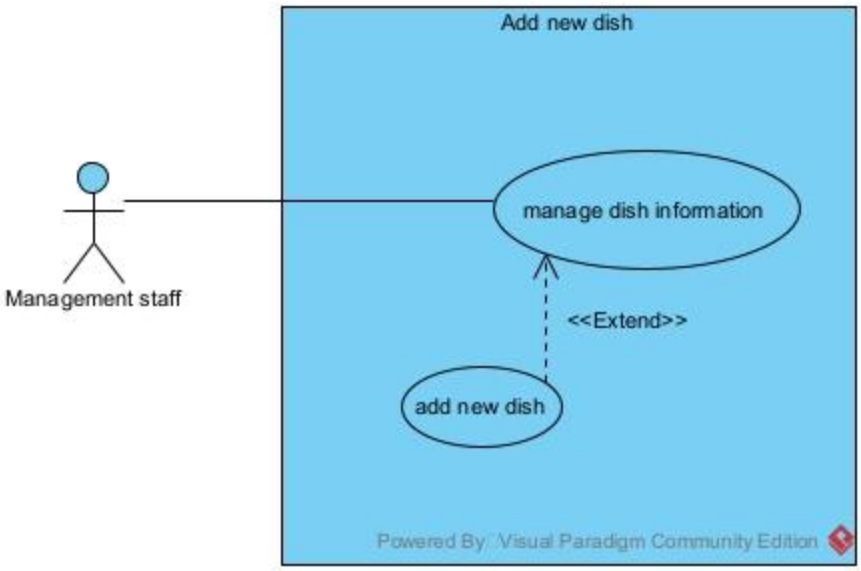


Description:

- Login: This use case enables the restaurant staff to log in to the restaurant system.
- View statistics: This use case enables the management staff to view the statistics of dishes, suppliers, or customer activities.

- Manage dish information: This use case enables the management staff to add new dishes, update existing dish information, or remove dishes from the system.
- Make combo menus: This use case enables the management staff to create, edit, or remove combo menus for customers.
- Manage supplier information: This use case enables the warehouse staff to add, edit, or remove supplier details from the system.
- Import materials: This use case enables the warehouse staff to record the import of materials from suppliers into the restaurant system.
- Receive customers: This use case enables the sales staff to welcome and serve customers when they arrive at the restaurant.
- Take orders: This use case enables the sales staff to take food and drink orders from customers at the restaurant.
- Receive payments: This use case enables the sales staff to process payments made by customers.
- Make membership cards: This use case enables the sales staff to register and issue membership cards to customers.
- Confirm reservation: This use case enables the sales staff to confirm table reservations made by customers.
- Confirm online orders: This use case enables the sales staff to verify and confirm online food orders placed by customers.
- Search: This use case enables customers to search for dishes, combo menus, or restaurant services.
- Book a table: This use case enables customers to reserve a table through the restaurant system.
- Order food online: This use case enables customers to place food orders online through the restaurant system.

b. Add a new dish

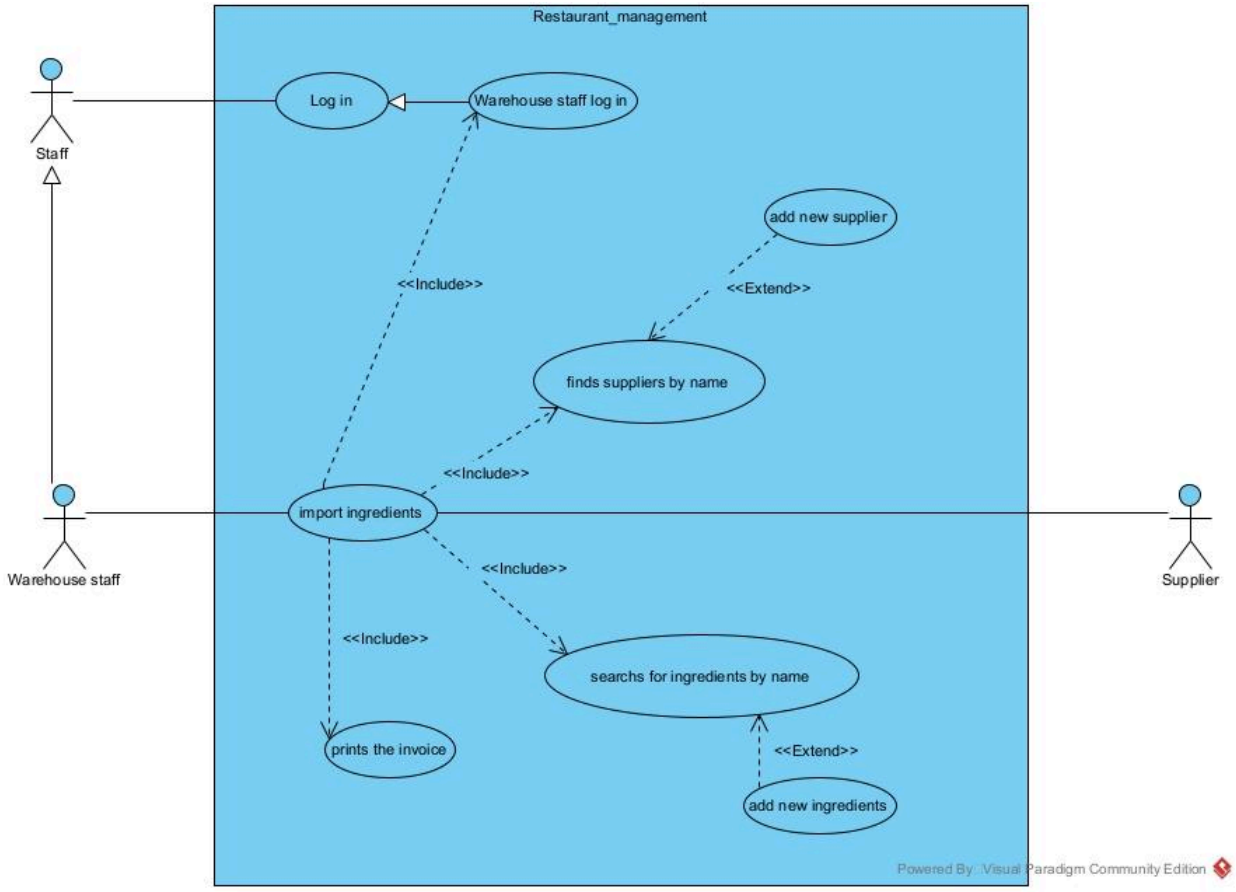


Description:

- Manage dish information: This use case enables the management staff to manage dish details in the restaurant system. The staff can add, update, or delete dish information to ensure the menu is up-to-date.

- Add new dish (Extend): This use allows the management staff to add a new dish into the system by entering dish name, category, ingredients, and price.

c. Import ingredients



Description:

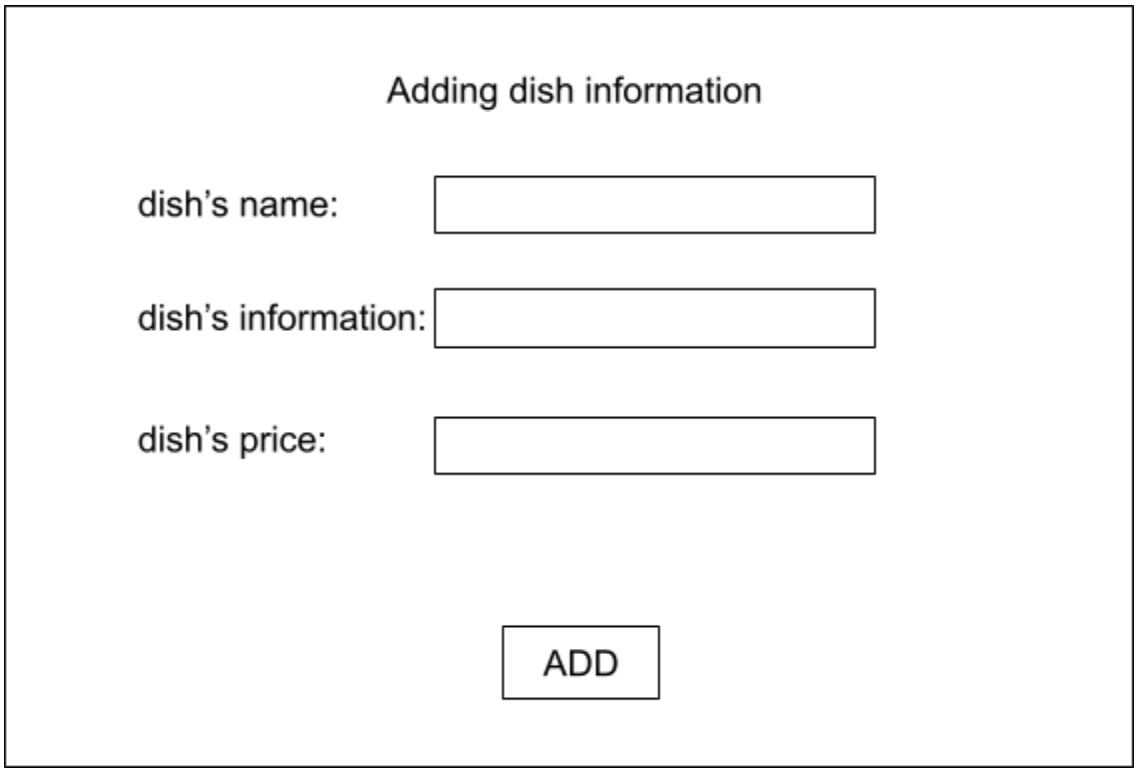
- Login: This use case enables the staff to log in to the restaurant management system.
- Warehouse staff log in: This use case specifies that the warehouse staff must log in successfully before performing warehouse-related functions.
- Import ingredients: This use case enables the warehouse staff to import ingredients into the restaurant system from suppliers. It includes searching for suppliers and ingredients, and printing invoices.
- Find suppliers by name: This use case enables the warehouse staff to search and find suppliers by their name in the system.
- Add new supplier: This use case extends the “Find suppliers by name” use case. If a supplier does not exist, the warehouse staff can add a new supplier into the system.
- Search for ingredients by name: This use case enables the warehouse staff to search ingredients by their name in the system before importing them.

- Add new ingredients: If an ingredient does not exist, the warehouse staff can add a new ingredient into the system.
- Print the invoice: This use case enables the warehouse staff to generate and print invoices for imported ingredients.

4. Scenario

a. Scenario for management staff adds new dish module

Use case	adds new dish
Actor	Management staff
Precondition	The management staff already has an account and logged in
Post condition	The management staff successfully added new dish
Main events	1.A management staff logins with username=”A” and password=”123” into the system 2.After successfully logged in, the system will display the main UI with three main function includes: view statistics, manage dish information and make combo menus 3.The management staff selects manage dish information 4.Another UI will be shown by the system, which has an option to select adding dish information function 5.The management staff clicks on adding dish information function 6. An interface pop up with 3 text boxes to fill include: dish name, dish’s information and dish’s price and an add button bellow

	 7. After filling the text boxes, the management staff clicks on the add button 8. The system will show the pop up announcement saying the dish information has been successfully added and the information will be saved to the database
Exceptions	2. The system shows that log in unsuccessfully (wrong password or wrong username) 7.1 The management staff clicks add without filling the text boxes or missing out on the boxes, the system will show a pop up message "Please fill in all the empty text box" 7.2 The management staff accidentally add a dish which has already existed in the database, then the system will show a pop up message "The dish is already added, please add another new dish"

b. scenario for warehouse staff imports ingredients

Use case	Imports ingredients
Actor	Warehouse staff
Precondition	The warehouse staff already has an account and logged in
Post condition	The warehouse staff successfully imported the ingredients then prints the invoice and pays to the supplier
Main events	1.A warehouse staff logins with username=”B” and password=”321” into the system 2.After successfully logged in, the system displays an UI with 2 functions: import ingredients and manage supplier information 3.The warehouse staff clicks on the import ingredients function 4.The system displays an interface consisting of a table of suppliers, a search bar and an “ADD NEW SUPPLIER” button. 5.The warehouse staff clicks on a supplier line 6.The system displays the ingredients interface including a search bar, a table(Ingredient name, unit, price per unit, an action button) and an “ADD NEW INGREDIENTS” button. Below the table is the “Confirm” button 7.The warehouse staff selects the ingredient from the result list and enters the quantity to import, repeats until all the ingredients are imported. 8.After finishing, the warehouse staff clicks on the confirm button, a display summary will be shown, consisting of supplier, ingredient list, quantities and total price. The interface also comes with a “Confirm import & print invoice” button. 9.The warehouse staff confirms the import order and clicks on the “Confirm import & print invoice” button 10.The system generates and prints the invoice 11.The warehouse staff proceeds with payments to the supplier
Exceptions	1. The system shows that log in unsuccessfully (wrong password or wrong username). 2. The warehouse staff clicks on the “ADD NEW SUPPLIER” button. - 2.1 An interface will pop up consist of text boxes (name, address, phone, email) and an “ADD” button - 2.2 After filling the text boxes, the warehouse staff clicks on the “ADD” button to create a new supplier line - 2.3 When adding a new supplier, if required data is missing or incorrectly formatted (missing name, invalid phone number), the system displays an error message and requests re-entry.

	3.The warehouse staff clicks on the “ADD NEW INGREDIENTS” button - 3.1 An interface will pop up consist of text boxes (name, address, phone, email) and an “ADD” button - 3.2 After filling the text boxes, the warehouse staff clicks on the “ADD” button to create a new ingredients line - 3.3 When adding a new ingredient, if the information is invalid (e.g., blank name, incorrect unit), the system does not save it and requests re-entry. 4. If the staff enters a supplier or ingredient that already exists in the database, the system displays a warning and suggests selecting from the list instead of creating a new one. 5. If the entered quantity is ≤ 0 or has the wrong format (letters instead of numbers), the system displays an error and requests re-entry 6. After confirming the import order, if the printer malfunctions, the system still saves the order and allows reprinting once the printer is operational again

3. Entity class

Step 1:

- ❖ Management staff: view statistics: dishes, ingredients, customers and suppliers. Manage dish information, make combo menus.
- ❖ Warehouse staff: import materials from suppliers, manage supplier information
- ❖ Sales staff: receive customers, take orders, receive payments at the table, make membership cards for customers, confirm table reservation information and order online.
- ❖ Customer: search, book a table and order food online.

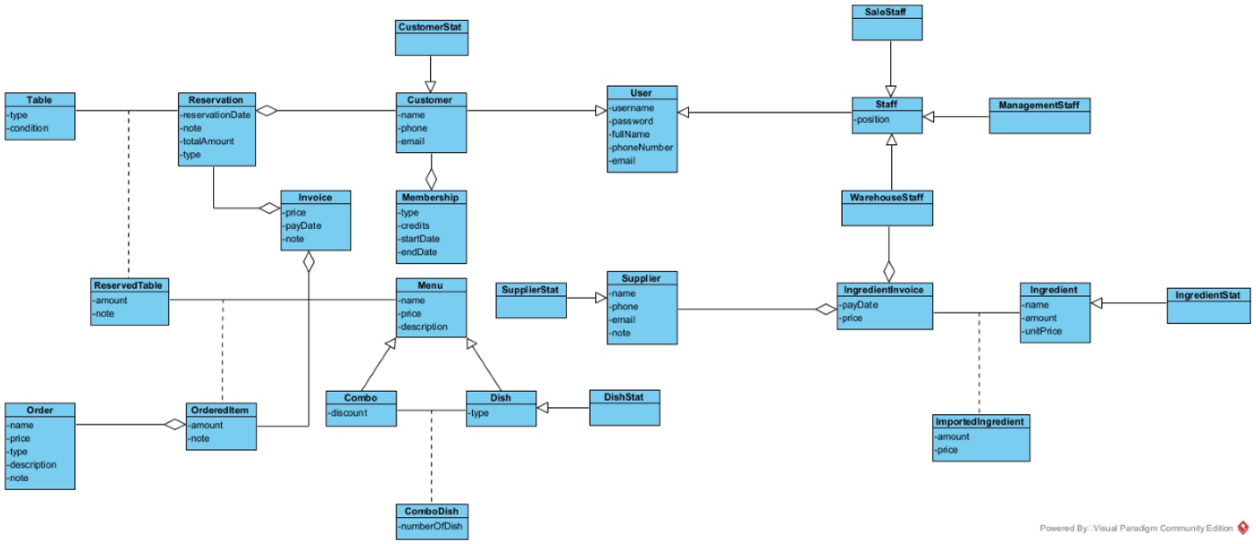
Step 2 + 3:

- ❖ Restaurant: Out of scope => eliminate
- ❖ System: too abstract => eliminate
- ❖ Management staff => Class ManagementStaff: username, password, name
- ❖ Sale staff => Class SaleStaff: username, password, name

- Warehouse staff => Class User: username, password, name
- Statistic: too abstract => eliminate
- Dishes => Class Dish
- Ingredients => Class Ingredient: name, amount, price
- Customer => Class Customer: name, phone, email
- Suppliers => Class Supplier: name, phone, email, note
- Information: too general => eliminate
- Combo => Class Combo
- Menu => Class Menu: name, price, description
- Order => ClassOrder: name, price, type, description, note
- Payment => Class Invoice: price, payDate, note
- Membership card => Class Membership: type, credits, startDate, endDate
- Table => Class Table: type, condition
- Reservation => Class Reservation: reservationDate, note, totalAmount
- Food: too general => eliminate

Step 4:

- Reservation - Customer: 1-n
- Reservation - Invoice: 1-n
- Reservation - Table: n-n => create a new class ReservedTable: amount, note
- Customer - Membership: 1-1
- ReservedTable - Menu: n-n => create a new class OrderedItem: amount, note
- Menu can be a Dish or a Combo => so Dish and Combo is inherited from Menu
- Combo - dish: n-n => create a new class ComboDish: numberOfDish
- Order - OrderItem: 1-n
-

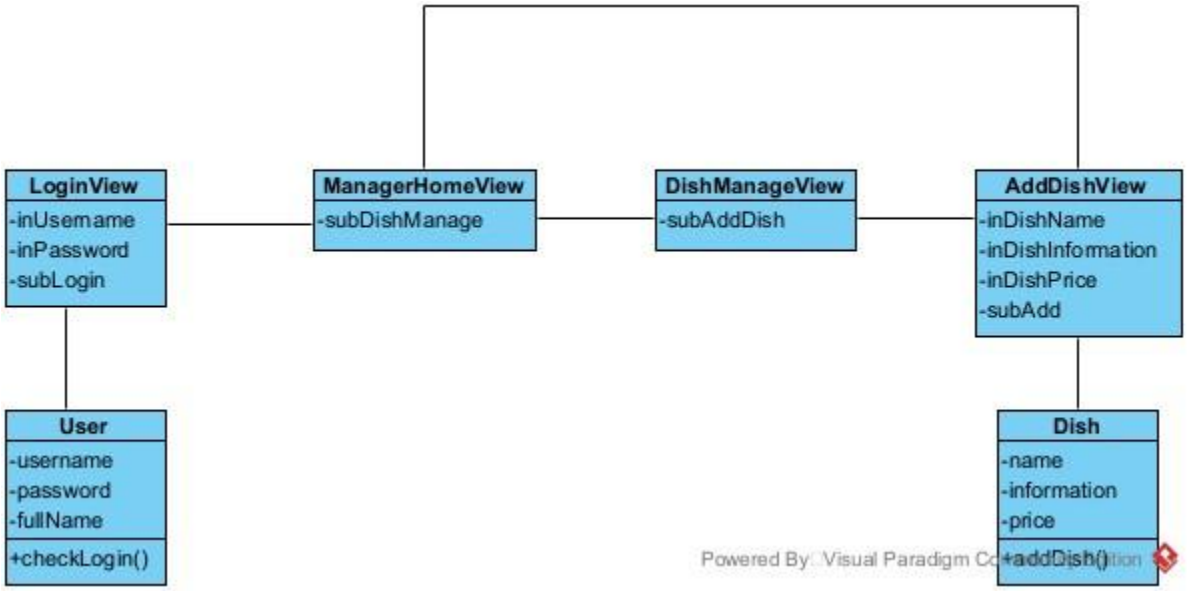


Class diagram for each module

1. **Module adds new dish**

- The management staff enter the system => The login interface appeared => need a class LoginView
    - Input for username => inUsername
    - Input for password => inPassword
    - A submit to login => subLogin
- Enter the username/password => the system must check if the login is correct => need a method:
    - name: checkLogin()
    - input: username, password(class of User)
    - output: boolean
    - assign to the entity class: User
- Once login is successful => the main interface of the manager is appeared => need a class ManagerHomeView which has a least:
    - an option to choose to manage dish information => subDishManage
- Choose the option to manage dish information => dish manage interface is appeared => need a class DishManageView which has at least:
    - an option to choose to add new dish => subAddDish

- Choose the option to add new dish => the add new dish interface is appeared => need a class AddDishView:
    - an input text to add the new dish name => inDishName
    - an input text to add the new dish information => inDishInformation
    - an input text to add the new dish price => inDishPrice
    - a submit to add the new dish => subAdd
- Add some new attributes and click add => The system has to update in to DB => need a method:
    - name: addDish()
    - input: an object of Dish
    - output: none or boolean
    - => assign to entity class: Dish
- After adding the system return to the ManagerHomeView

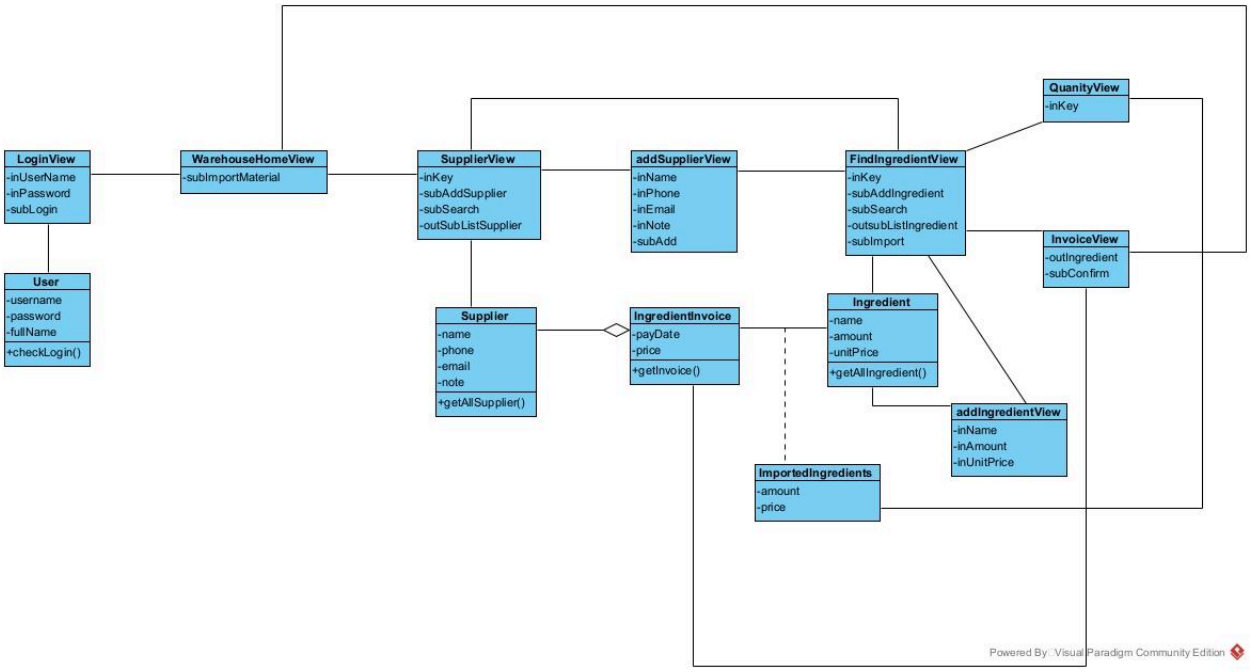


2. Module import

- The warehouse staff enter the system => The login interface appeared => need a class LoginView
    - Input for username => inUsername
    - Input for password => inPassword
    - A submit to login => subLogin
- Enter the username/password => the system must check if the login is correct => need a method:

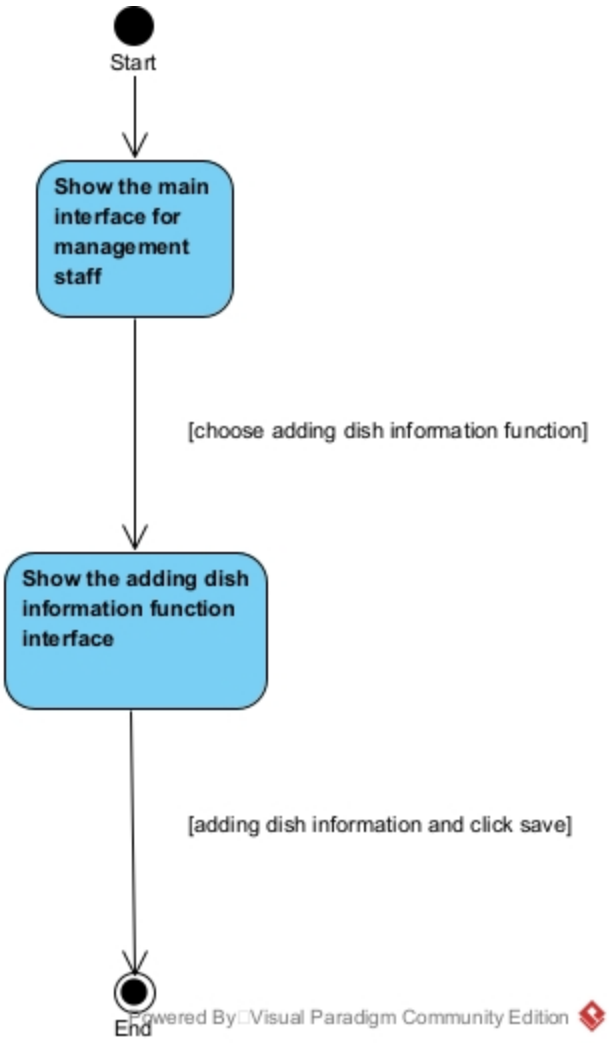
    - name: checkLogin()
    - input: username, password(class of User)
    - output: boolean
    - assign to the entity class: User
- Once login is successful => the main interface of the warehouse staff appears => need a class WarehouseHomeView which has at least:
    - an option to choose import ingredients => subImportMaterial
- When the warehouse staff wants to import ingredients => the warehouse staff choose the import option => the interface to find the supplier appeared => need a class: SupplierView:
    - an input text to enter the keyword(search supplier by name) => inKey
    - a button to add new supplier(in case of new supplier) => subAddSupplier
    - a search button => subSearch
    - a result table which could be clicked on to choose a supplier => outsubListSupplier
- Display a list of supplier => need a method
    - name: getAllSupplier()
    - input: none
    - output: a list of supplier
    - => assign to the entity class: Supplier
- The list supplier will be listed in the SupplierView
- Click on a supplier to view the ingredient list (in case of a new supplier, choose to add a new supplier and the addSupplierView appears, that needs a method to add the client into the DB).
- The interface to find ingredient is appeared => need a class FindIngredientView
    - an input text to enter the keyword(search ingredient by name) => inKey
    - a button to add new ingredient(in case of new supplier) => subAddIngredient
    - a search button => subSearch
    - a result table which could be clicked on to choose an ingredient=> outsubListIngredient
    - a button to import the ingredient => subImport

- Display a list of ingredient => need a method
    - name: getAllIngredient()
    - input: none
    - output: a list of ingredient
    - => assign to the entity class: IngredientInvoice
- The warehouse staff choose the correct ingredient to import(when it comes to enter quantity, the QuantityView appears. In case of new ingredient, choose to add new ingredient and the addIngredientView appeared, that needs a method to add new ingredient into the DB)
- The warehouse staff choose to import the ingredients after choosing => the Invoice interface is appeared => need a class InvoiceView
    - display all information about the selected Ingredient => outIngredient
    - a confirm button => subConfrim
- Display the invoice => need a method
    - name: getInvoice()
    - input: none
    - output: an invoice
    - => assign to the entity class: Invoice
- The warehouse staff choose to confirm the Invoice => The system prints out the invoice

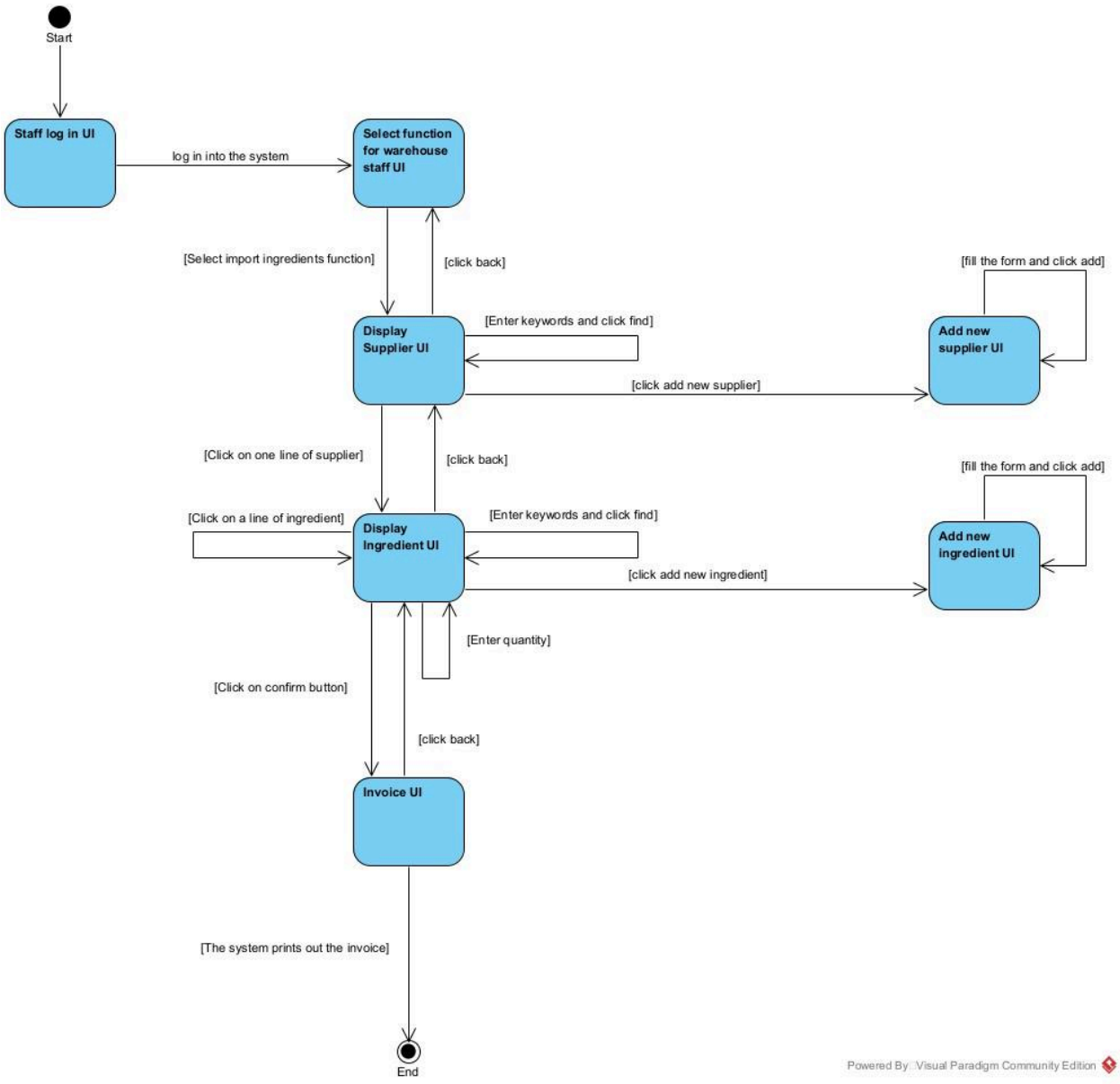


State diagram

- Module 1:
    - After entering username/password and select login, the system will display the interface for the management staff
    - From the main interface, the staff select the adding dish information and the system will display the interface for adding dish information
    - After entering dishes information and click Add, the system will save the information into the database and finish



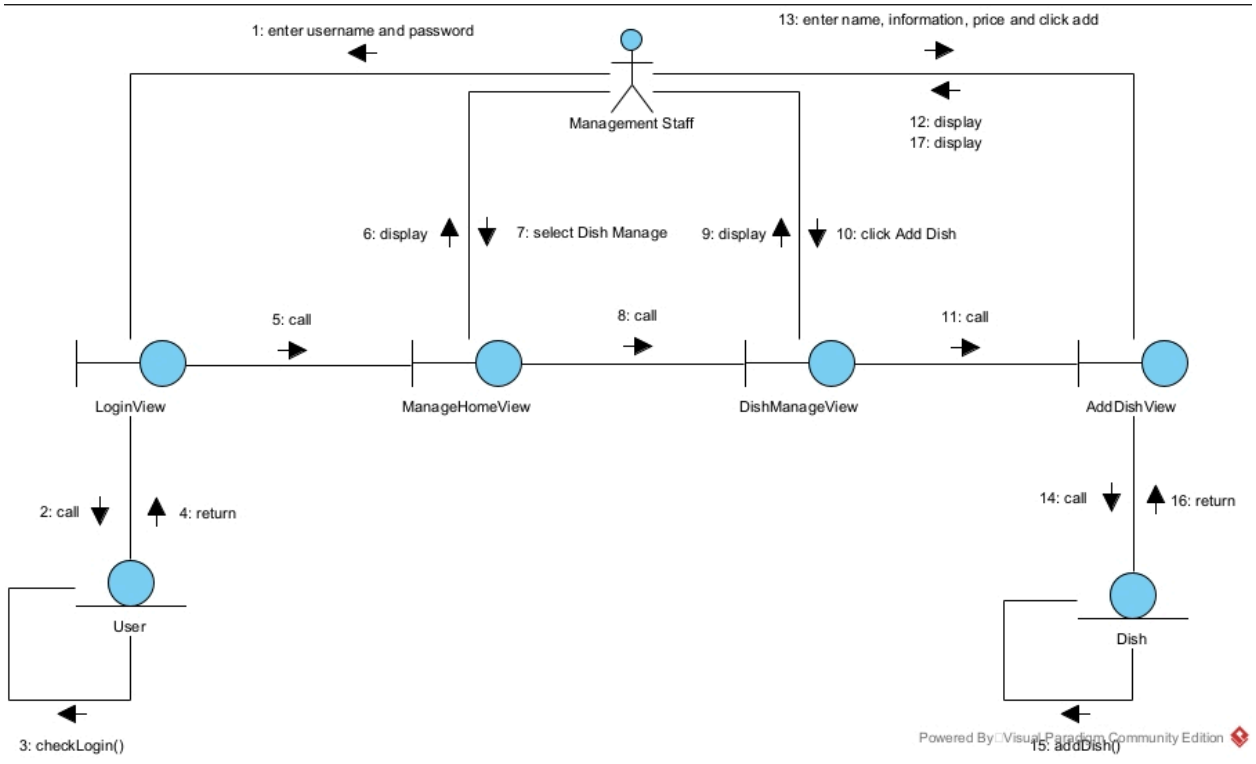
- Module 2:
    - The staff log in the system after entering username
    - After logging in, the system displays Warehouse staff UI with functions to choose
    - Select on the import ingredients function, the system display the supplier UI
    - Click on a line of customer, the system display the Ingredient UI
    - Click on a line of ingredient and enter quantity and click on confirm button, the system display the Invoice UI
    - Select the print button, the system prints out the invoice and finish



Communication diagram

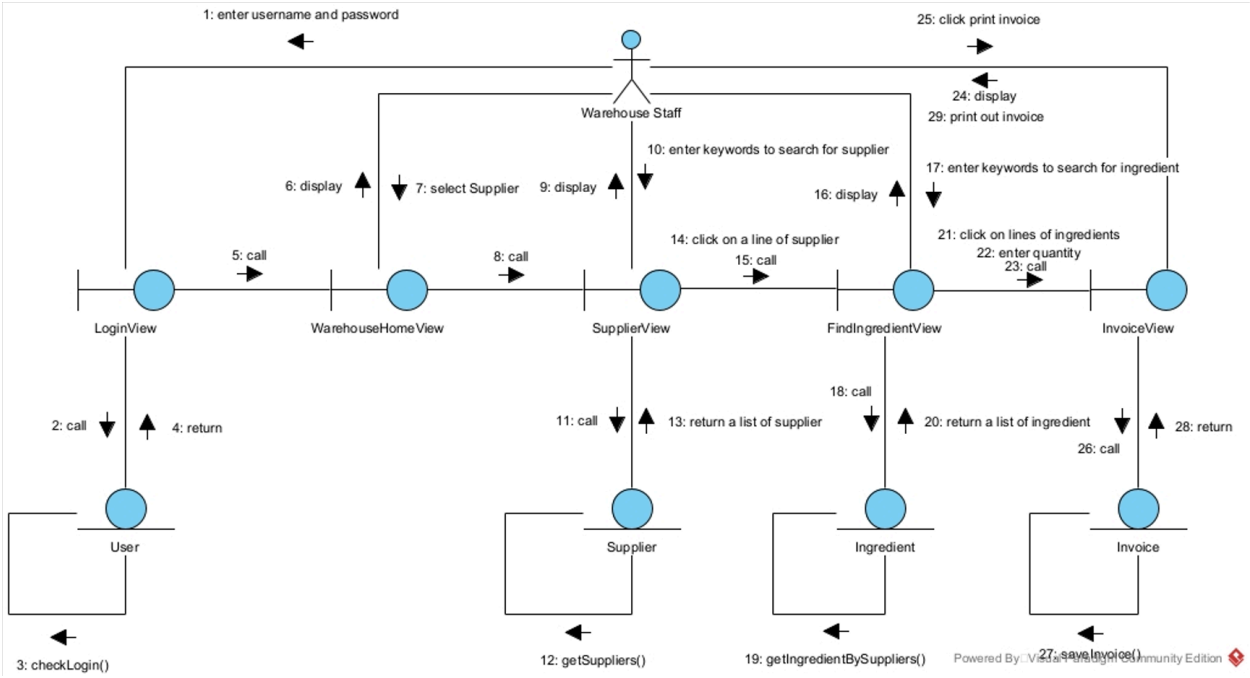
- Module 1:
    - The management staff enters username/password and clicks Login button
    - The LoginView call class User to proceed
    - The class User calls the method checkLogin()
    - The class Staff returns the results to the class LoginView
    - The class LoginView calls the class ManagerHomeView
    - The class ManagerHomeView display itself to the management staff

- The management staff select Dish Manage button
- The class ManagerHomeView call the class DishManageView
- The class DishManageView display itself to the management staff
- The management staff select the AddDish button
- The class DishManageView calls the class AddDishView
- The class AddDishView display itself to the management staff
- The management staff enters the dish's informations
- The class AddDishView calls class Dish to proceed
- The class Dish calls the method addDish()
- The class Dish returns the results to the class AddDishView
- The class AddDishView display the notification and itself for the management staff



- Module 2:
  - The warehouse staff enters username/password and clicks Login button
  - The class LoginView calls the class User to process
  - The class User returns the results to the class LoginView
  - The class LoginView calls the class WarehouseHomeView

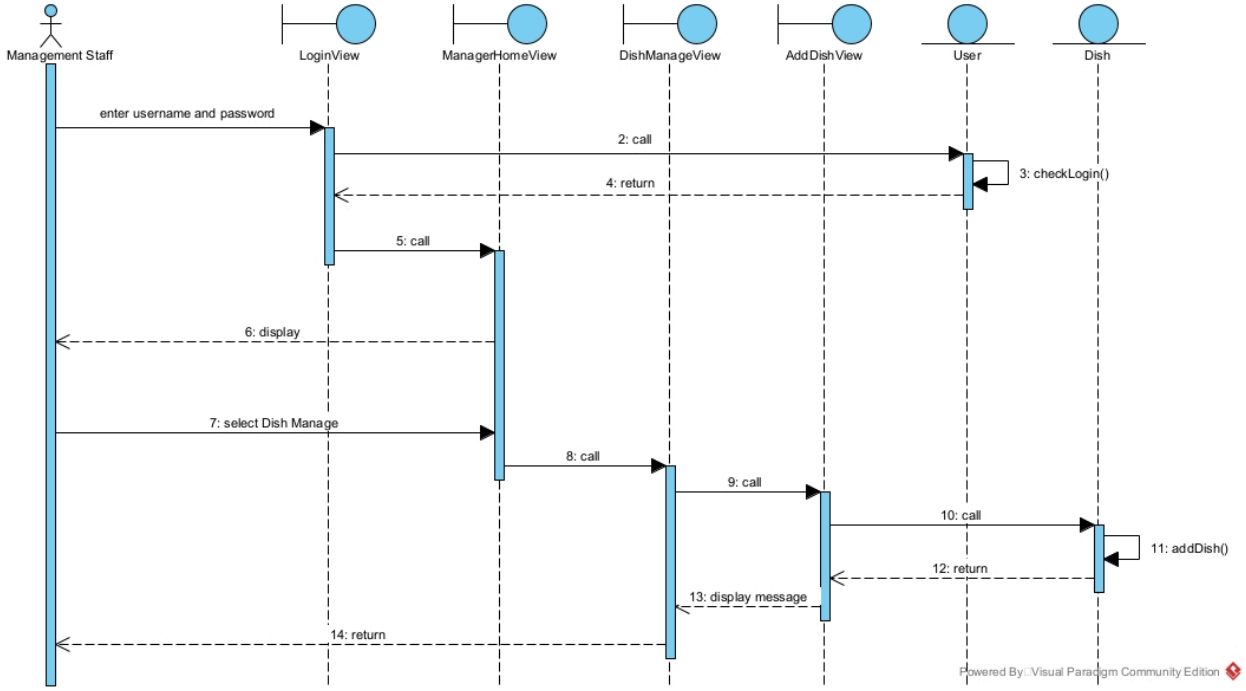
- The WarehouseHomeView display itself to the management staff
- The management staff select ImportIngredient function
- The WarehouseHomeView call the SupplierView
- The SupplierView display itself to the WarehouseStaff
- The WarehouseStaff enter keywords to search for supplier
- The SupplierView calls the class Supplier
- The class Supplier calls the method getAllSupplier()
- The class Supplier returns the list of suppliers to the class SupplierView
- The WarehouseStaff clicks on a line of supplier and the class SupplierView calls the class FindIngredientView
- The class FindIngredientView display itself to the WarehouseStaff
- The Warehouse Staff enters keywords to search for ingredients
- The FindIngredientView call the class Ingredient
- The class Ingredient calls the method getAllIngredient()
- The class Ingredient returns the list of ingredients to the class FindIngredientView
- The Warehouse Staff click on lines of ingredients, enter quantities and then click on the Confirm button
- The class FindIngredientView call the class InvoiceView
- The class InvoiceView class display itself to the Warehouse Staff
- The Warehouse Staff click on the Print Invoice button
- The InvoiceView call the class Invoice
- The class Invoice calls the method getInvoice()
- The class Invoice returns the invoice to the InvoiceView
- The InvoiceView display the invoice and the system prints out the invoice for the Warehouse Staff



Scenario diagram

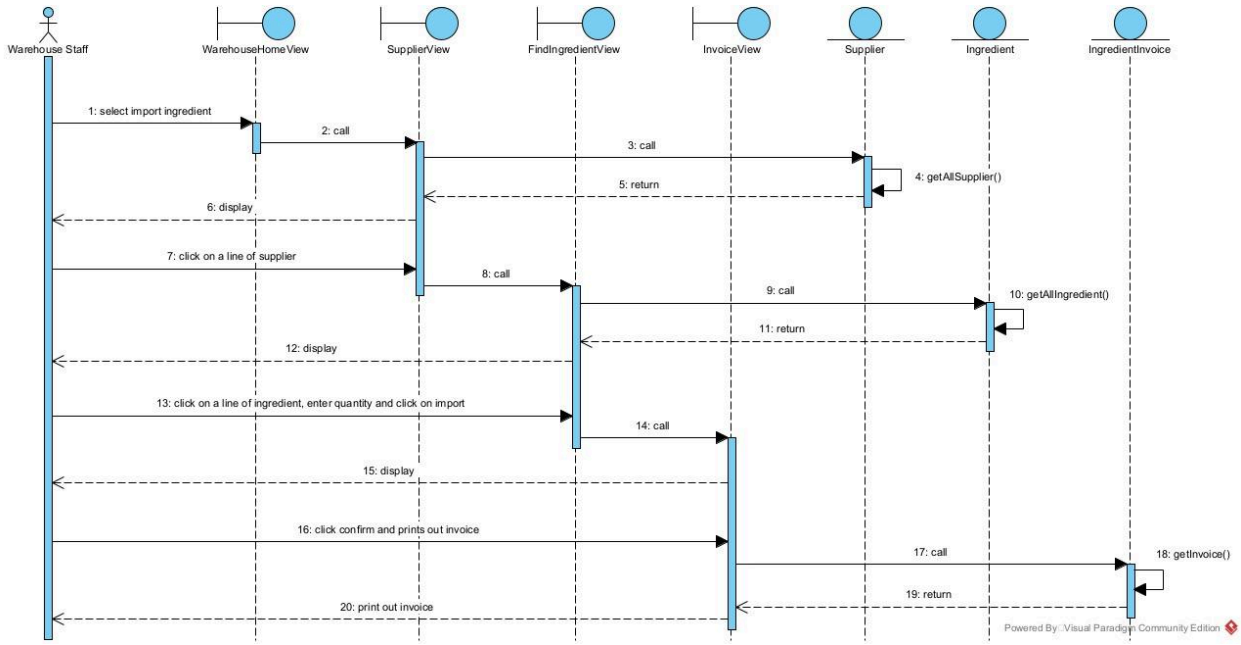
- Module 1:
    1. The Management Staff enters username and password in the LoginView interface.
    2. The LoginView interface calls the User class.
    3. The User class calls its own checkLogin() method.
    4. The User class returns the result to the LoginView interface.
    5. The LoginView interface calls the ManagerHomeView interface.
    6. The ManagerHomeView interface displays itself to the Management Staff.
    7. The Management Staff selects "Dish Manage" on the ManagerHomeView interface.
    8. The ManagerHomeView interface calls the DishManageView interface.
    9. The DishManageView interface calls the AddDishView interface.
    10. The AddDishView interface calls the Dish class.
    11. The Dish class calls its own addDish() method.
    12. The Dish class returns the result to the AddDishView interface.

13. The AddDishView interface sends a "display message" call to the DishManageView interface.
14. The DishManageView interface returns to the ManagerHomeView interface.



- Module 2:
    1. The Warehouse Staff selects "import ingredient" on the WarehouseHomeView interface.
    2. The WarehouseHomeView interface calls the SupplierView interface.
    3. The SupplierView interface calls the Supplier class.
    4. The Supplier class calls its own getAllSupplier() method.
    5. The Supplier class returns the result to the SupplierView interface.
    6. The SupplierView interface displays the supplier list to the Warehouse Staff.
    7. The Warehouse Staff clicks on a line of a supplier in the SupplierView interface.
    8. The SupplierView interface calls the FindIngredientView interface.

9. The FindIngredientView interface calls the Ingredient class.
10. The Ingredient class calls its own getAllIngredient() method.
11. The Ingredient class returns the result to the FindIngredientView interface.
12. The FindIngredientView interface displays the ingredient list to the Warehouse Staff.
13. The Warehouse Staff clicks on a line of an ingredient, enters quantity, and clicks "import" on the FindIngredientView interface.
14. The FindIngredientView interface calls the InvoiceView interface.
15. The InvoiceView interface displays itself to the Warehouse Staff.
16. The Warehouse Staff clicks "confirm and prints out invoice" on the InvoiceView interface.
17. The InvoiceView interface calls the IngredientInvoice class.
18. The IngredientInvoice class calls its own getInvoice() method.
19. The IngredientInvoice class returns the result to the InvoiceView interface.
20. The InvoiceView interface prints out the invoice for the Warehouse Staff.



III. Design

1. Add ID attribute to non-inheriting classes

- Table: id : in
- Reservation: id : int
- ReservedTable: id : int
- Order: id : int
- OrderedItem: id : int
- Invoice: id : int
- Membership: id : int
- Menu: id : int
- Combo: id : int
- ComboDish: id : int
- Dish: id : int
- CustomerStat: id : int
- Customer: id : int
- User: id : int
- Staff: id : int
- SaleStaff: id : int
- ManagementStaff: id : int
- WarehouseStaff: id : int
- SupplierStat: id : int
- Supplier: id : int
- IngredientInvoice: id : int
- ImportedIngredient: id : int
- Ingredient: id : int
- IngredientStat: id : int

2. Add type for attributes

- Table
    - id : int
    - type : string
    - condition : string

- Reservation
    - id : int
    - reservationDate : date
    - note : string
    - totalAmount : int
    - type : string
- ReservedTable
    - id : int
    - amount : int
    - note : string
- Order
    - id : int
    - name : string
    - price : float
    - type : string
    - description : string
    - note : string
- OrderedItem
    - id : int
    - amount : int
    - note : string
- Invoice
    - id : int
    - price : float
    - payDate : date
    - note : string
- Membership
    - id : int
    - type : string
    - credits : int
    - startDate : date
    - endDate : date
- Menu
    - id : int

    - name : string
    - price : float
    - description : string
- Combo
    - id : int
    - discount : float
- ComboDish
    - id : int
    - numberOfDish : int
- Dish
    - id : int
    - type : string
- CustomerStat
    - id : int

- Customer
    - id : int
    - name : string
    - phone : string
    - email : string
- User
    - id : int
    - username : string
    - password : string
    - fullName : string
    - phoneNumber : string
    - email : string
- Staff
    - id : int
    - position : string
- SaleStaff
    - id : int
- ManagementStaff
    - id : int
- WarehouseStaff

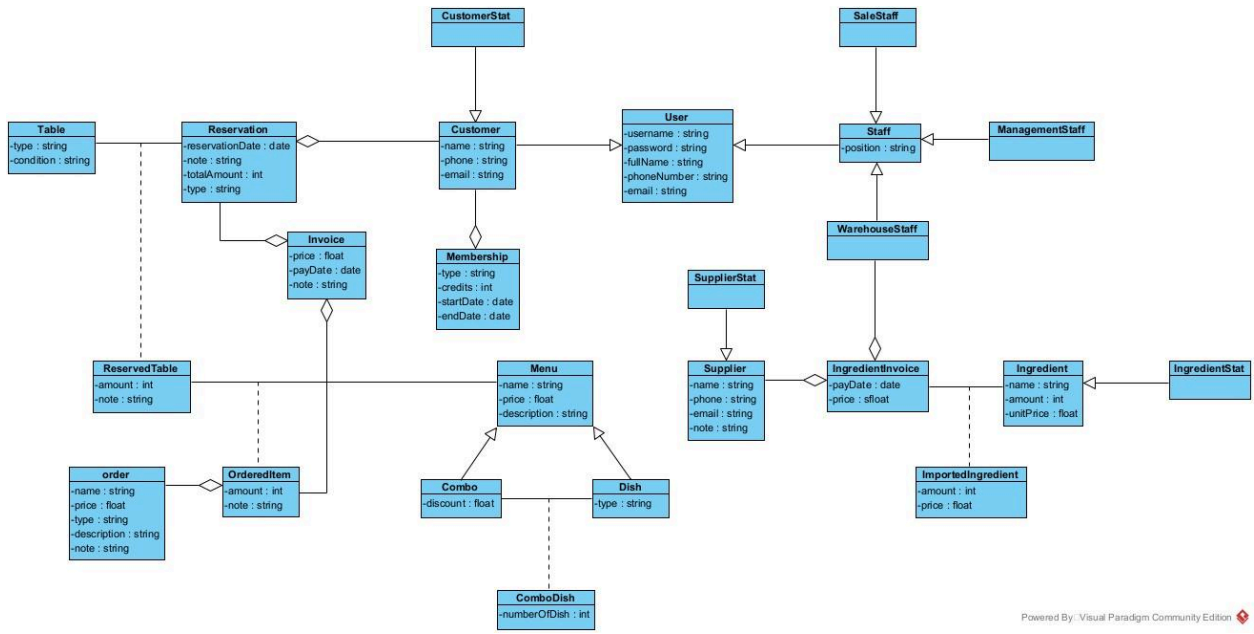
    - id : int
- SupplierStat
    - id : int
- Supplier
    - id : int
    - name : string
    - phone : string
    - email : string
    - note : string
- IngredientInvoice
    - id : int
    - payDate : date
    - price : float
- ImportedIngredient
    - id : int
    - amount : int
    - price : float
- Ingredient
    - id : int
    - name : string
    - amount : int
    - unitPrice : float
- IngredientStat

    - id : int

3. Add object attributes from the aggregation/composition relationship

- ReservedTable is a component of Reservation (1–N) → Reservation has a list of ReservedTable
- Invoice is a component of Reservation (1–1) → Reservation has Invoice
- Customer is a component of Invoice (1–N) → Invoice has Customer
- Membership is a component of Invoice (1–1) → Invoice has Membership
- OrderedItem is a component of Order (1–N) → Order has a list of OrderedItem
- Menu is a component of Combo (1–N) → Combo has Menu

● Dish is a component of Menu (1–N) → Menu has a list of Dish
● ComboDish is a component of Combo (N–1) → Combo has a list of ComboDish
● Dish is a component of ComboDish (1–N) → ComboDish has a list of Dish
● User is a component of Customer (1–1) → Customer has User
● SaleStaff is a specialization of Staff (1–1) → Staff generalizes SaleStaff
● ManagementStaff is a specialization of Staff (1–1) → Staff generalizes ManagementStaff
● WarehouseStaff is a specialization of Staff (1–1) → Staff generalizes WarehouseStaff
● IngredientInvoice is a component of WarehouseStaff (1–N) → WarehouseStaff has a list of IngredientInvoice
● ImportedIngredient is a component of IngredientInvoice (1–N) → IngredientInvoice has a list of ImportedIngredient
● Ingredient is a component of ImportedIngredient (1–1) → ImportedIngredient has Ingredient
● Ingredient is a component of Supplier (1–N) → Supplier has a list of Ingredient



Database design

Step 1: Entity class → corresponding table

- Class Table → tblTable
- Class Reservation → tblReservation
- Class ReservedTable → tblReservedTable
- Class Order → tblOrder
- Class OrderedItem → tblOrderedItem
- Class Invoice → tblInvoice
- Class Menu → tblMenu
- Class Combo → tblCombo
- Class ComboDish → tblComboDish
- Class Dish → tblDish
- Class Customer → tblCustomer
- Class User → tblUser
- Class Staff → tblStaff
- Class SaleStaff → tblSaleStaff
- Class ManagementStaff → tblManagementStaff
- Class WarehouseStaff → tblWarehouseStaff
- Class Supplier → tblSupplier
- Class IngredientInvoice → tblIngredientInvoice
- Class ImportedIngredient → tblImportedIngredient
- Class Ingredient → tblIngredient

Step 2: Move non-object properties to tables

- tblTable: id, type, condition
- tblReservation: id, reservationDate, totalAmount, type, note, tblCustomerId
- tblReservedTable: id, amount, note, tblTableId, tblReservationId
- tblOrder: id, name, price, type, description, note
- tblOrderedItem: id, amount, note, tblReservedTableId, tblOrderId, tblMenuId
- tblInvoice: id, price, payDate, note, tblOrderedItemId, tblReservationId
- tblMenu: id, name, price, description
- tblCombo: id, discount, tblMenuId
- tblComboDish: id, numberOfDish, tblComboId, tblDishId, tblMenuId
- tblDish: id, type, tblMenuId
- tblCustomer: id, name, phone, email, tblUserId

- tblUser: id, username, password, fullName, phoneNumber, email
- tblStaff: id, position, tblUserId
- tblSaleStaff: id, tblStaffId
- tblManagementStaff: id, tblStaffId
- tblWarehouseStaff: id, tblStaffId
- tblSupplier: id, name, phone, email, note
- tblIngredientInvoice: id, payDate, price, tblSupplierId, tblWarehouseStaffId
- tblImportedIngredient: id, amount, price, tblIngredientInvoiceId, tblIngredientId
- tblIngredient: id, name, amount, unitPrice

Step 3: Determine quantity relationships

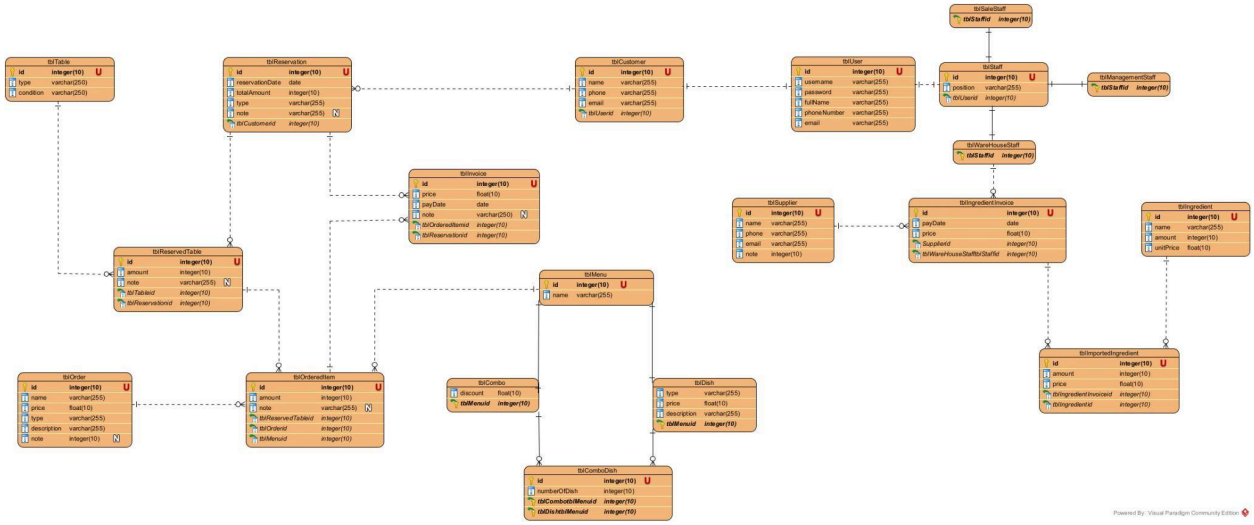
- 1 tblCustomer – N tblReservation
- 1 tblReservation – N tblReservedTable
- 1 tblReservation – N tblInvoice
- 1 tblReservedTable – N tblOrderedItem
- 1 tblOrder – N tblOrderedItem
- 1 tblMenu – N tblOrderedItem
- 1 tblMenu – N tblCombo
- 1 tblMenu – N tblDish
- 1 tblCombo – N tblComboDish
- 1 tblDish – N tblComboDish
- 1 tblUser – 1 tblCustomer / tblStaff
- 1 tblStaff – 1 tblSaleStaff / tblManagementStaff / tblWarehouseStaff
- 1 tblSupplier – N tblIngredientInvoice
- 1 tblWarehouseStaff – N tblIngredientInvoice
- 1 tblIngredientInvoice – N tblImportedIngredient
- 1 tblIngredient – N tblImportedIngredient

Step 4: Define keys for tables

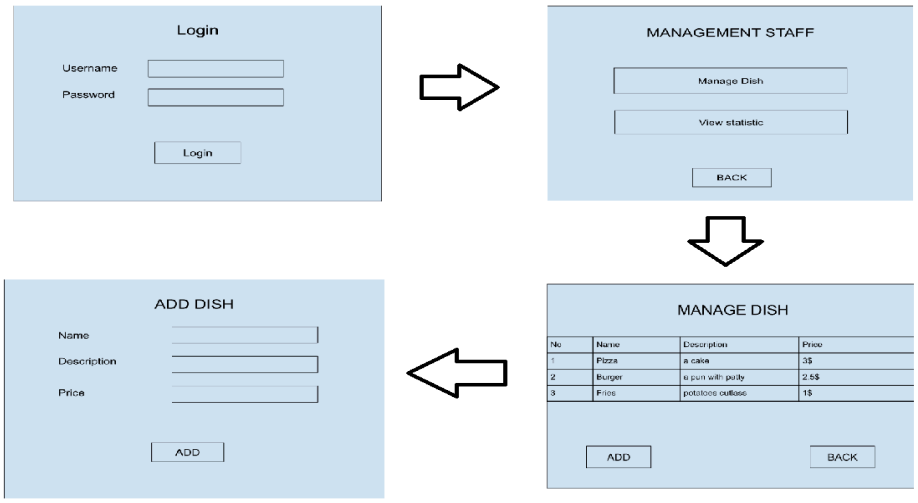
- Primary Keys: every table has an id column as the PK
- Foreign Keys:
    - 1 tblCustomer – N tblReservation → table tblReservation has foreign key tblCustomerId

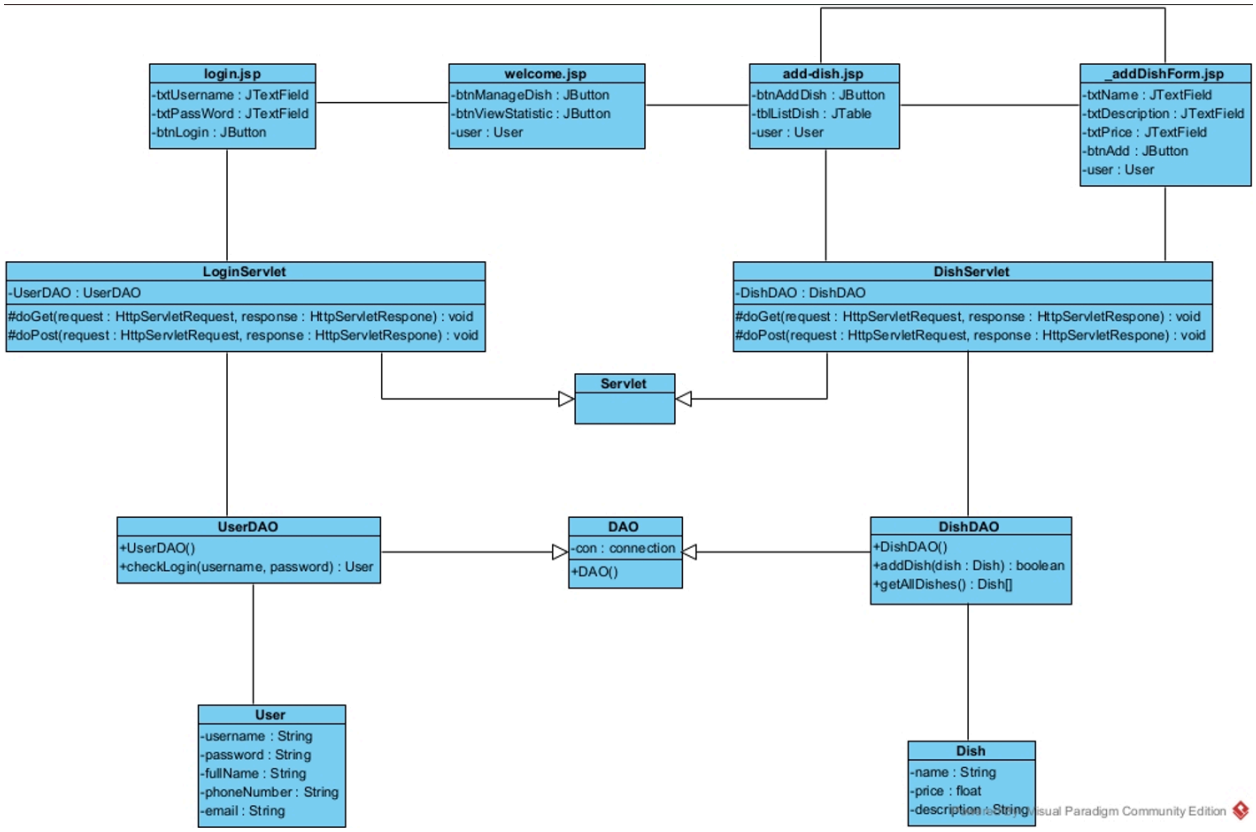
○ 1 tblReservation – N tblReservedTable → table tblReservedTable has foreign key tblReservationId
○ 1 tblTable – N tblReservedTable → table tblReservedTable has foreign key tblTableId
○ 1 tblReservation – N tblInvoice → table tblInvoice has foreign key tblReservationId
○ 1 tblReservedTable – N tblOrderedItem → table tblOrderedItem has foreign key tblReservedTableId
○ 1 tblOrder – N tblOrderedItem → table tblOrderedItem has foreign key tblOrderId
○ 1 tblMenu – N tblOrderedItem → table tblOrderedItem has foreign key tblMenuId
○ 1 tblMenu – N tblCombo → table tblCombo has foreign key tblMenuId
○ 1 tblMenu – N tblDish → table tblDish has foreign key tblMenuId
○ 1 tblCombo – N tblComboDish → table tblComboDish has foreign key tblComboId
○ 1 tblDish – N tblComboDish → table tblComboDish has foreign key tblDishId
○ 1 tblUser – 1 tblCustomer → table tblCustomer has foreign key tblUserId
○ 1 tblUser – 1 tblStaff → table tblStaff has foreign key tblUserId
○ 1 tblStaff – 1 tblSaleStaff → table tblSaleStaff has foreign key tblStaffId
○ 1 tblStaff – 1 tblManagementStaff → table tblManagementStaff has foreign key tblStaffId
○ 1 tblStaff – 1 tblWarehouseStaff → table tblWarehouseStaff has foreign key tblStaffId
○ 1 tblSupplier – N tblIngredientInvoice → table tblIngredientInvoice has foreign key tblSupplierId
○ 1 tblWarehouseStaff – N tblIngredientInvoice → table tblIngredientInvoice has foreign key tblWarehouseStaffId
○ 1 tblIngredientInvoice – N tblImportedIngredient → table tblImportedIngredient has foreign key tblIngredientInvoiceId

○ 1 tblIngredient – N tblImportedIngredient → table tblImportedIngredient has foreign key tblIngredientId

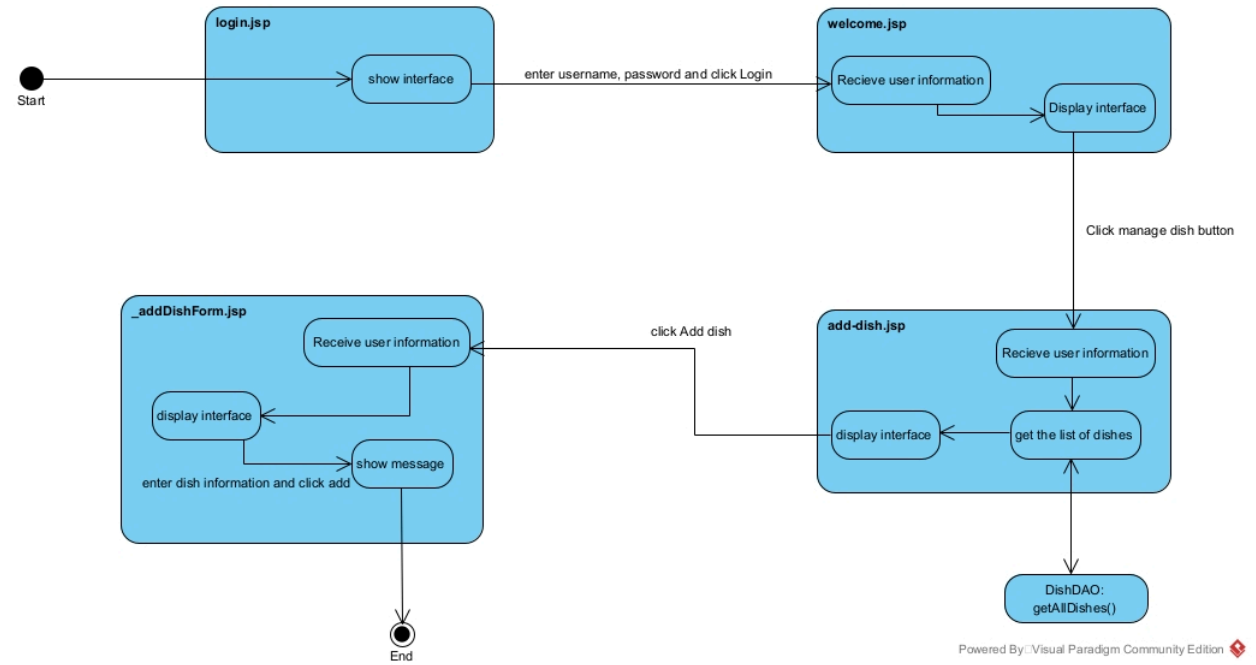


Module 1:
Class diagram:





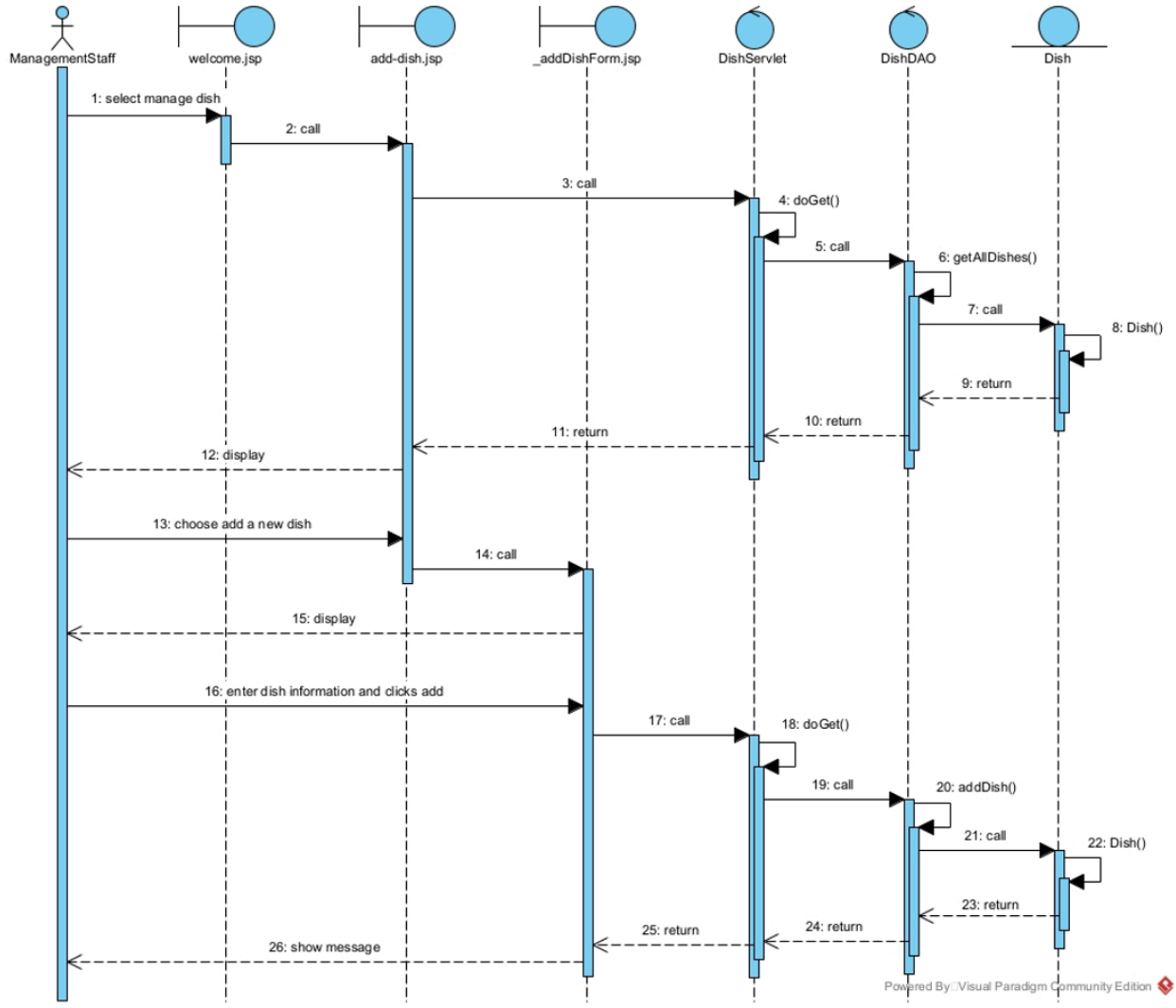
Activity diagram:



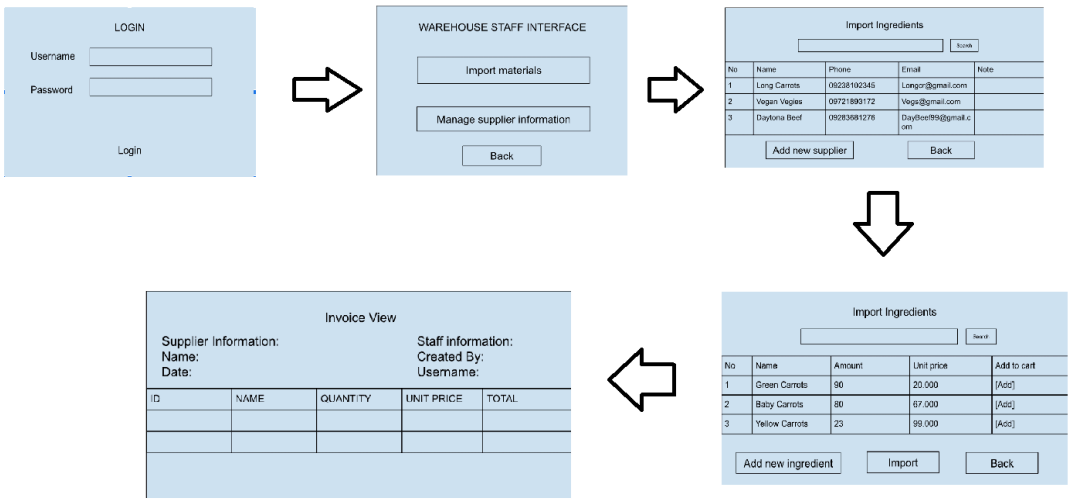
Scenario v3:

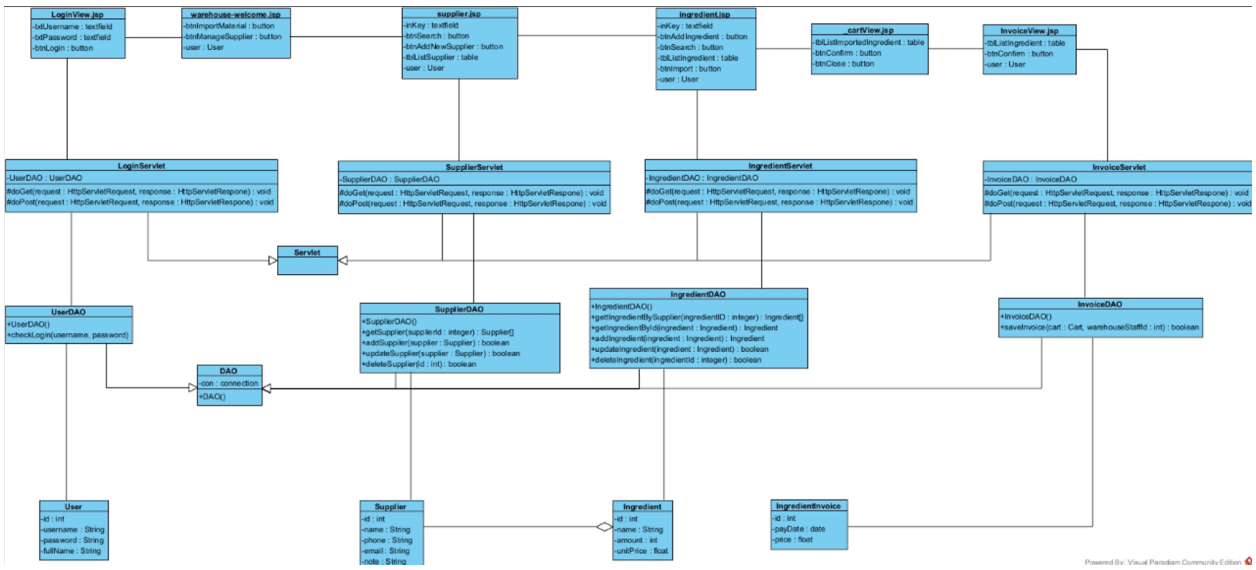
1. The ManagementStaff selects "manage dish" in the welcome.jsp interface.
2. The welcome.jsp interface calls the add-dish.jsp interface.

3. The add-dish.jsp interface calls the DishServlet class.
4. The DishServlet class calls its own doGet() method.
5. The doGet() method calls the DishDAO class.
6. The DishDAO class calls its getAllDishes() method.
7. The getAllDishes() method calls the Dish class.
8. The Dish class returns Dish objects to the getAllDishes() method.
9. The getAllDishes() method returns the result to the doGet() method.
10. The doGet() method returns the result to the DishServlet class.
11. The DishServlet class returns the result to the add-dish.jsp interface.
12. The add-dish.jsp interface displays the list of dishes to the ManagementStaff.
13. The ManagementStaff chooses "add a new dish" on the add-dish.jsp interface.
14. The add-dish.jsp interface calls the _addDishForm.jsp interface.
15. The _addDishForm.jsp interface displays itself to the ManagementStaff.
16. The ManagementStaff enters dish information and clicks "add" on the _addDishForm.jsp interface.
17. The _addDishForm.jsp interface calls the DishServlet class.
18. The DishServlet class calls its own doGet() method.
19. The doGet() method calls the DishDAO class.
20. The DishDAO class calls its addDish() method.
21. The addDish() method calls the Dish class.
22. The Dish class returns a Dish object to the addDish() method.
23. The addDish() method returns the result to the doGet() method.
24. The doGet() method returns the result to the DishServlet class.
25. The DishServlet class returns the result to the _addDishForm.jsp interface.
26. The _addDishForm.jsp interface shows a message to the ManagementStaff.

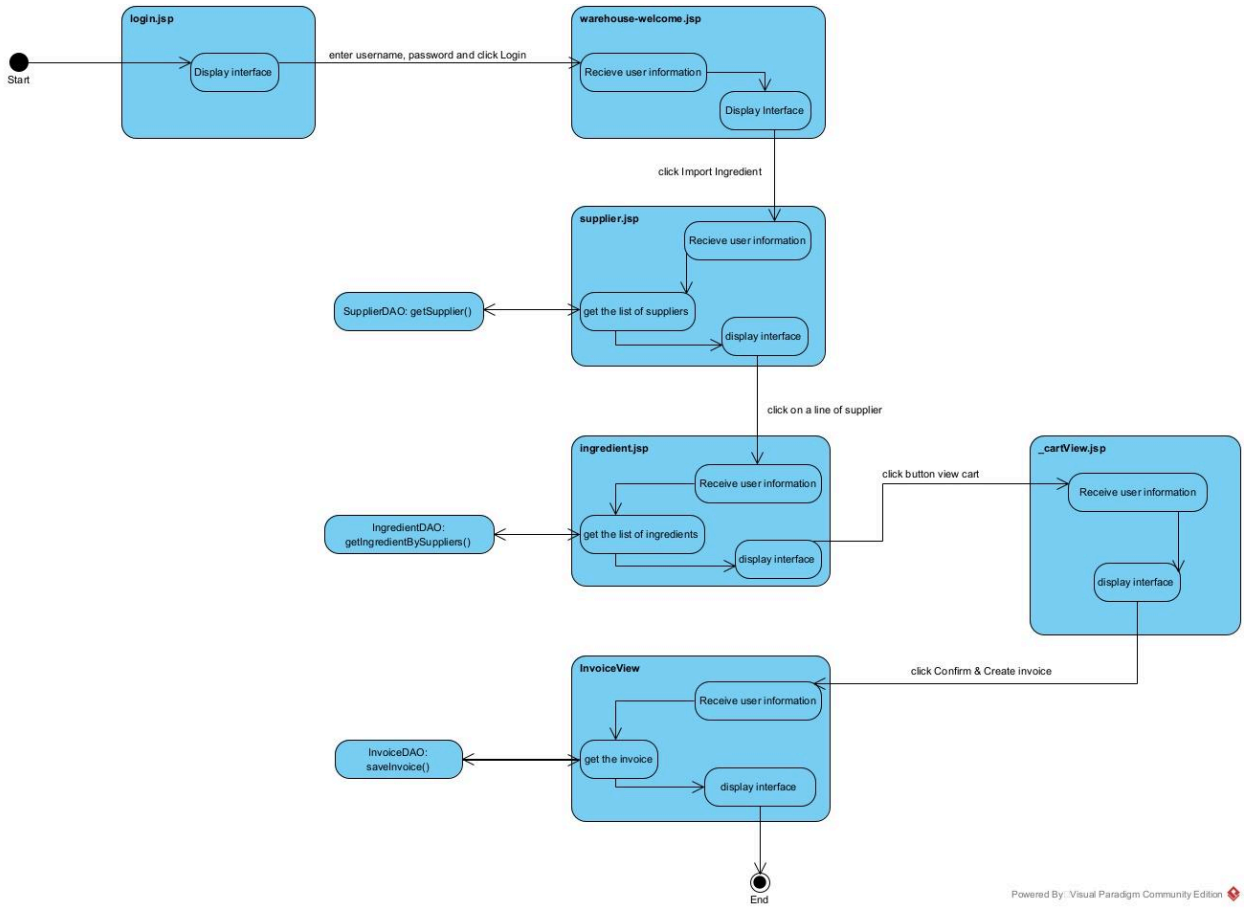


Module 2:
Class Diagram:





Activity diagram:

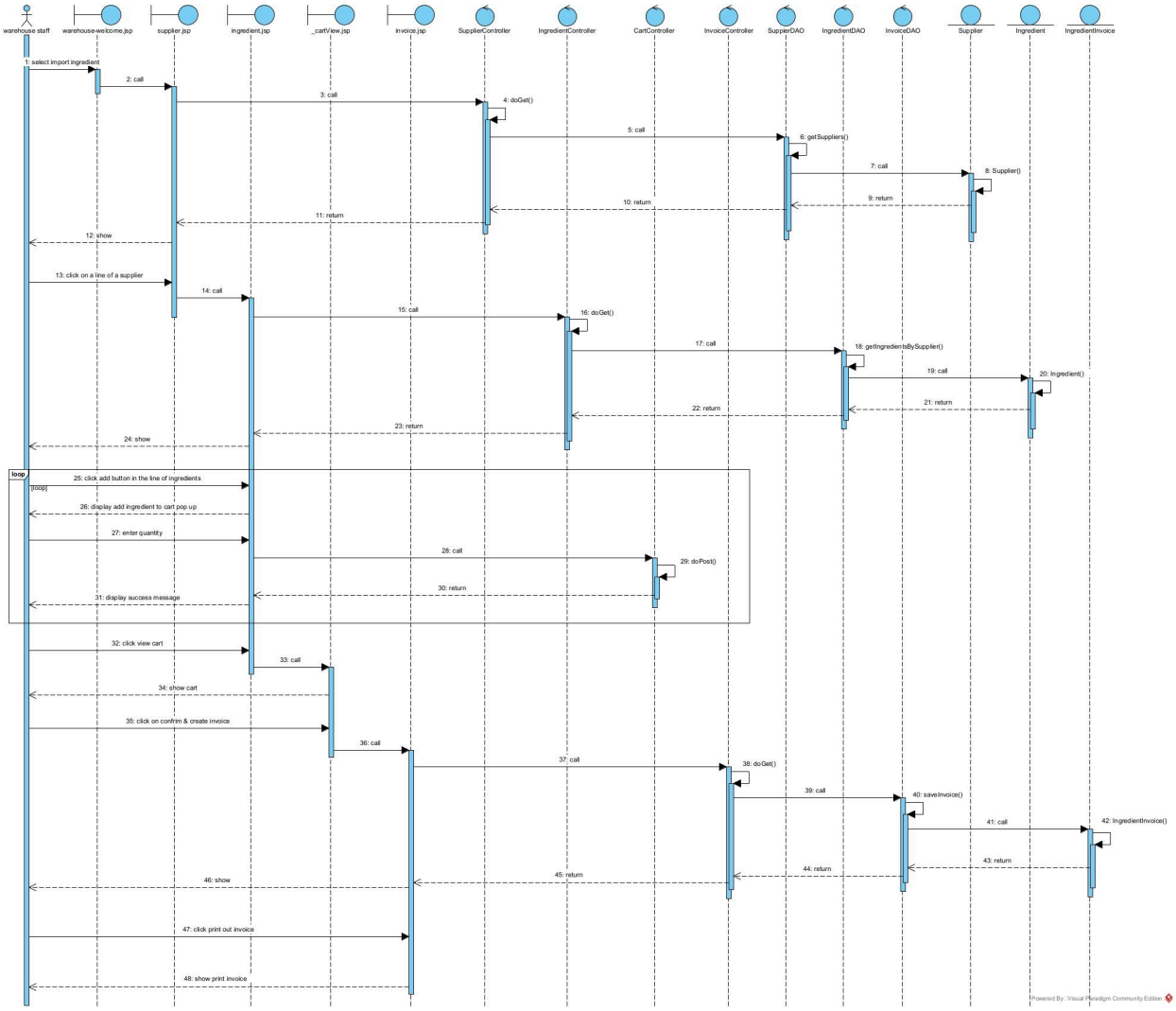


Sequence diagram:

1. The warehouse staff selects "import ingredient" in the warehouse.welcome.jsp interface.
2. The warehouse.welcome.jsp interface calls the supplier.jsp interface.
3. The supplier.jsp interface calls the SupplierController class.
4. The SupplierController class calls its own doGet() method.
5. The doGet() method calls the SupplierDAO class.
6. The SupplierDAO class calls its getSuppliers() method.
7. The getSuppliers() method calls the Supplier class.
8. The Supplier class returns supplier objects to the getSuppliers() method.
9. The getSuppliers() method returns the result to the doGet() method.
10. The doGet() method returns the result to the supplier.jsp interface.
11. The supplier.jsp interface returns the result to the warehouse.welcome.jsp interface.
12. The warehouse.welcome.jsp interface shows the suppliers to the warehouse staff.

13. The warehouse staff clicks on a supplier line in the warehouse.welcome.jsp interface.
14. The warehouse.welcome.jsp interface calls the supplier.jsp interface.
15. The supplier.jsp interface calls the IngredientController class.
16. The IngredientController class calls its own doGet() method.
17. The doGet() method calls the IngredientDAO class.
18. The IngredientDAO class calls its getIngredientBySupplier() method.
19. The getIngredientBySupplier() method calls the Ingredient class.
20. The Ingredient class returns ingredient objects to the getIngredientBySupplier() method.
21. The getIngredientBySupplier() method returns the result to the doGet() method.
22. The doGet() method returns the result to the supplier.jsp interface.
23. The supplier.jsp interface returns the result to the warehouse.welcome.jsp interface.
24. The warehouse.welcome.jsp interface shows the ingredients to the warehouse staff.
25. The warehouse staff clicks the "add" button in the line of an ingredient on the warehouse.welcome.jsp interface.
26. The warehouse.welcome.jsp interface displays an "add ingredient to cart" pop-up to the warehouse staff.
27. The warehouse staff enters a quantity on the warehouse.welcome.jsp interface.
28. The warehouse.welcome.jsp interface calls the CartController class.
29. The CartController class calls its own doPost() method.
30. The doPost() method returns a result to the warehouse.welcome.jsp interface.
31. The warehouse.welcome.jsp interface displays a success message to the warehouse staff.
32. The warehouse staff clicks "view cart" on the warehouse.welcome.jsp interface.
33. The warehouse.welcome.jsp interface calls the cartView.jsp interface.
34. The cartView.jsp interface shows the cart to the warehouse staff.
35. The warehouse staff clicks "confirm & create invoice" on the cartView.jsp interface.

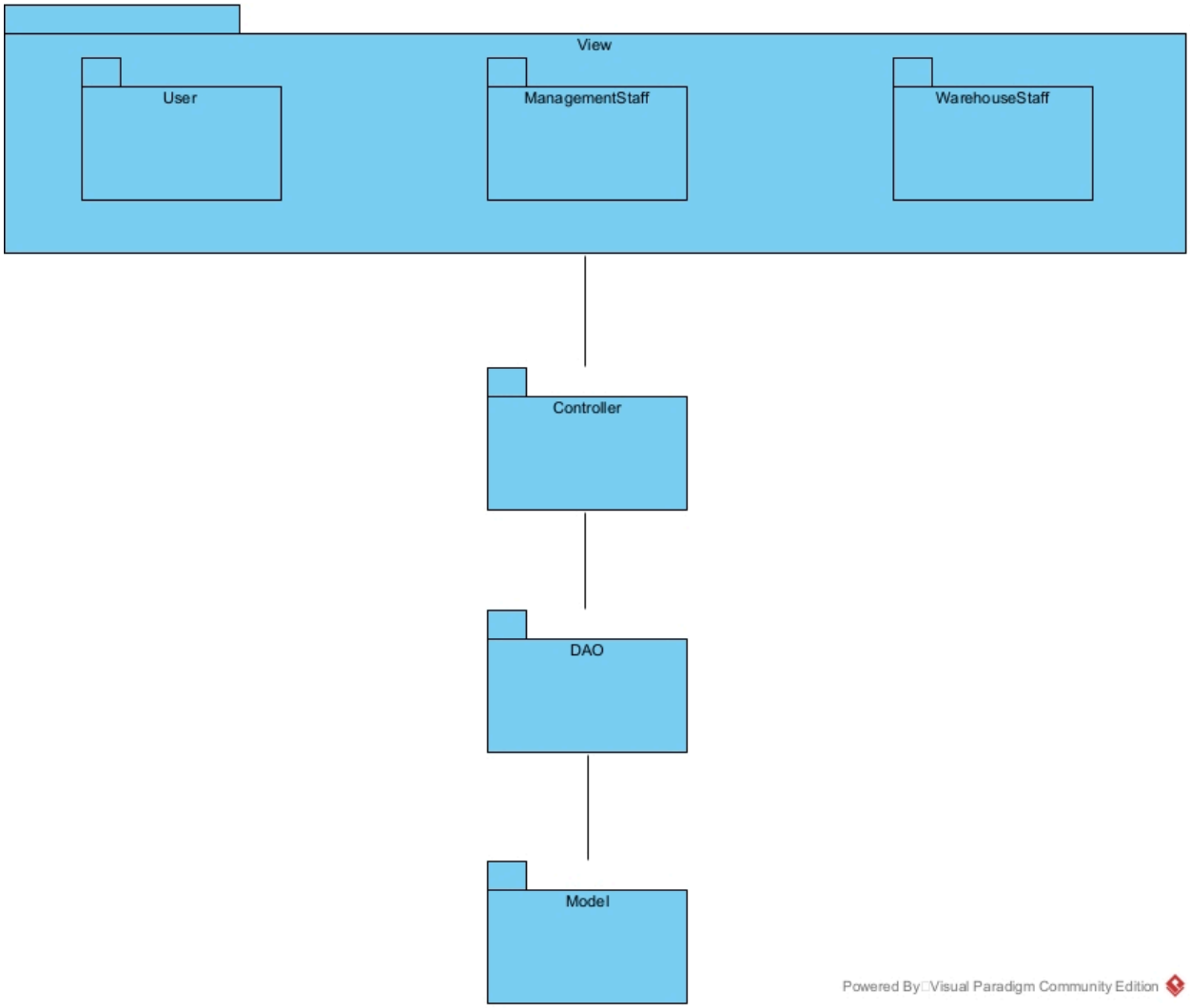
36. The cartView.jsp interface calls the InvoiceController class.
37. The InvoiceController class calls its own doGet() method.
38. The doGet() method calls the InvoiceDAO class.
39. The InvoiceDAO class calls its saveInvoice() method.
40. The saveInvoice() method calls the IngredientInvoice class.
41. The IngredientInvoice class returns an object to the saveInvoice() method.
42. The saveInvoice() method returns the result to the doGet() method.
43. The doGet() method returns the result to the cartView.jsp interface.
44. The cartView.jsp interface calls the invoice.jsp interface.
45. The invoice.jsp interface shows the invoice to the warehouse staff.
46. The warehouse staff clicks "print out invoice and pay the supplier" on the invoice.jsp interface.
47. The invoice.jsp interface shows the print button to the warehouse staff.



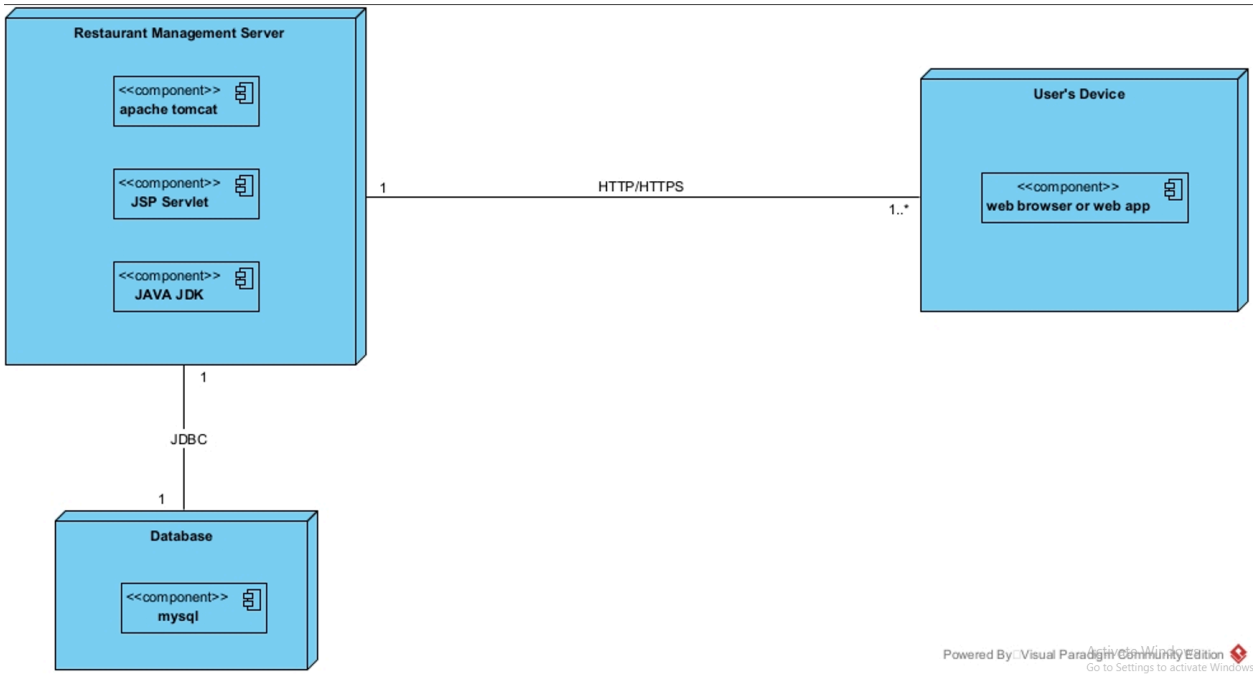
Package diagram

The entity classes are placed together in the model package
The DAO classes are placed together in the DAO package
The JSP pages are placed in the view package. The view package is further divided into smaller subpackages corresponding to the interfaces for different types of users:

- Pages related to login operations are places in the user package
- Pages related to Management Staff function are places in the Management staff package
- Pager related to Warehouse Staff function are placed in Warehouse Staff package



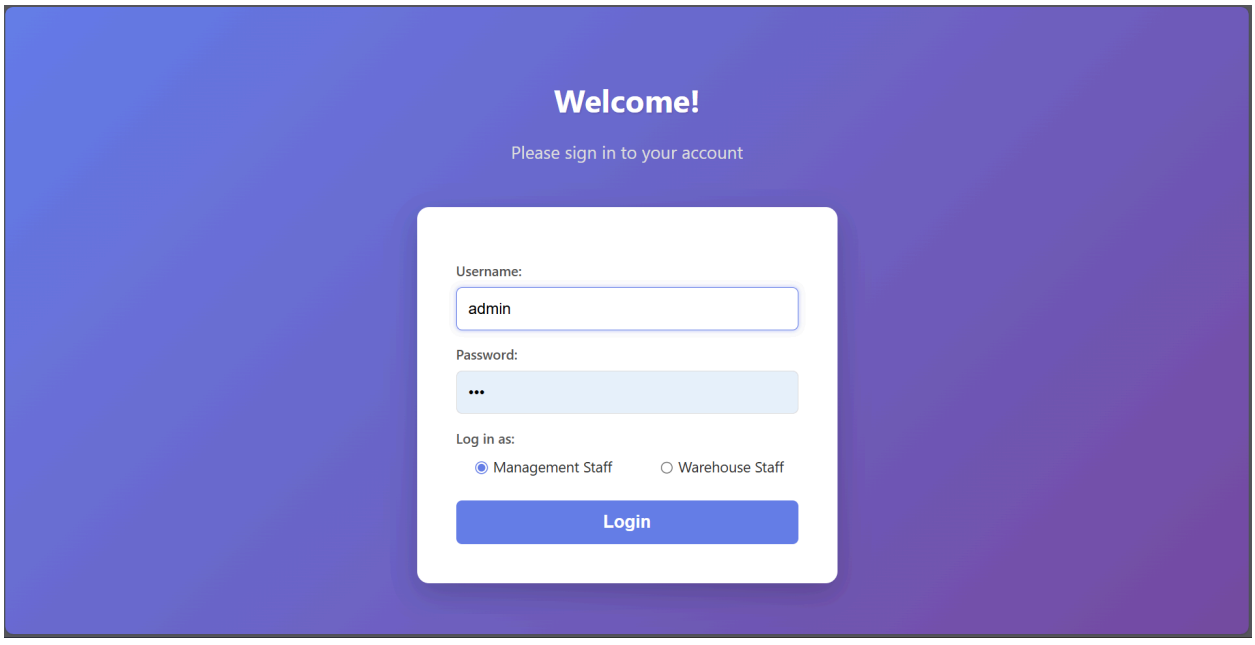
Deployment diagram:



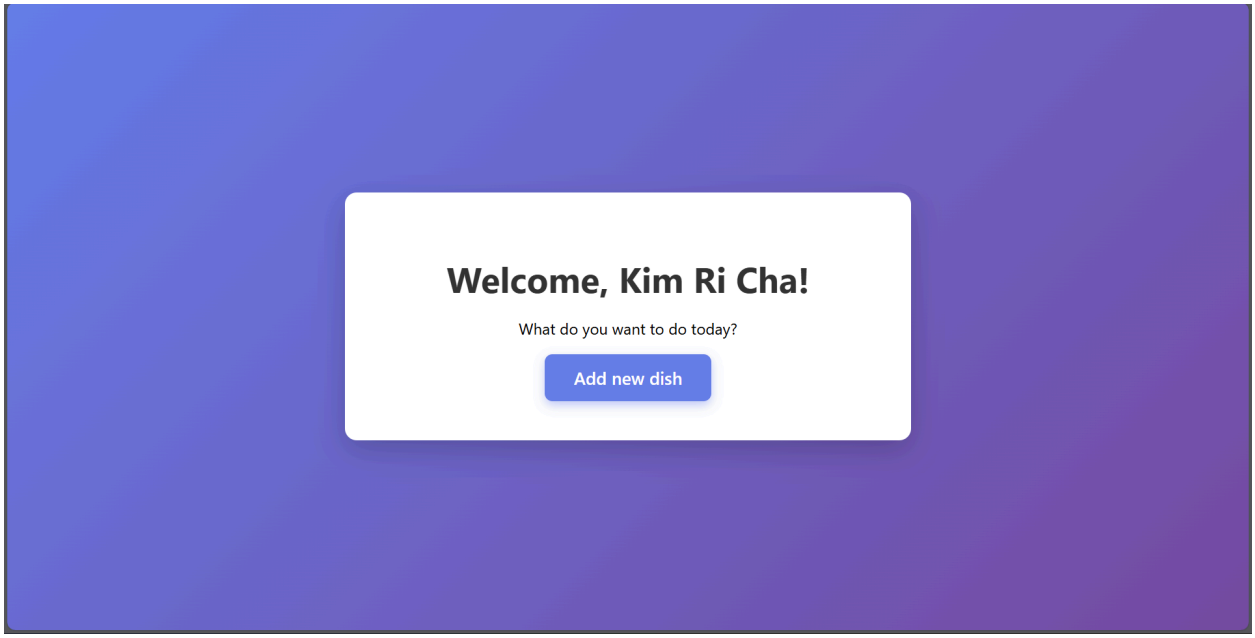
IV. Programing

1. UI

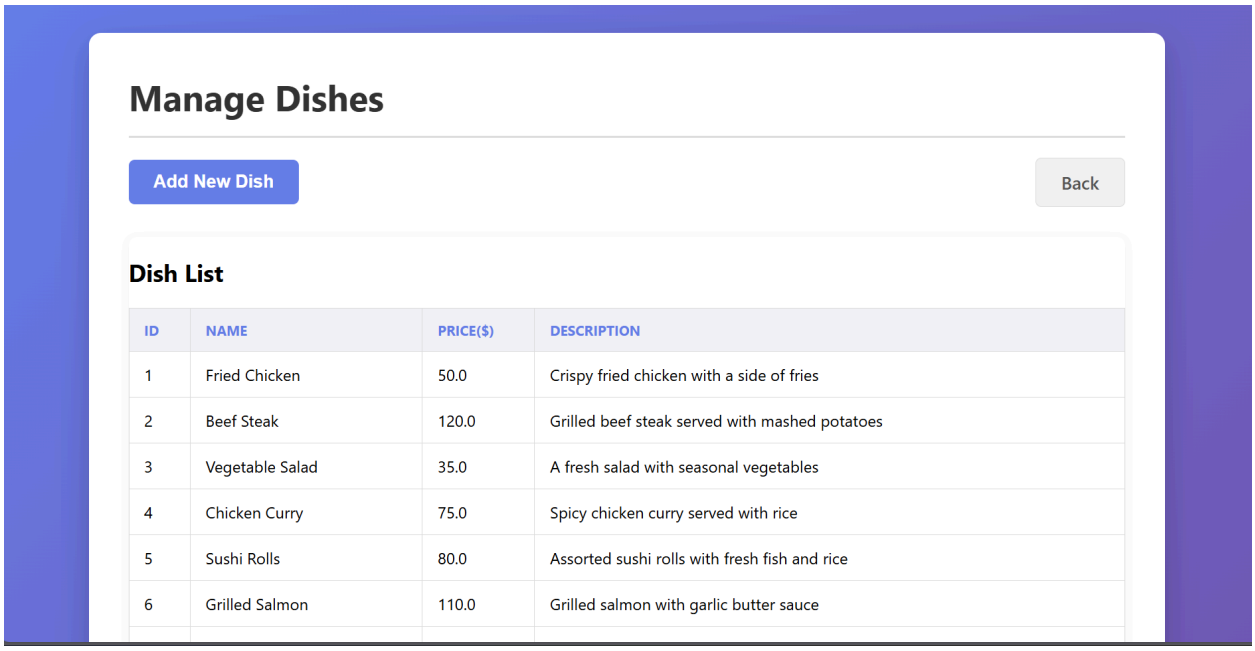
- login.jsp



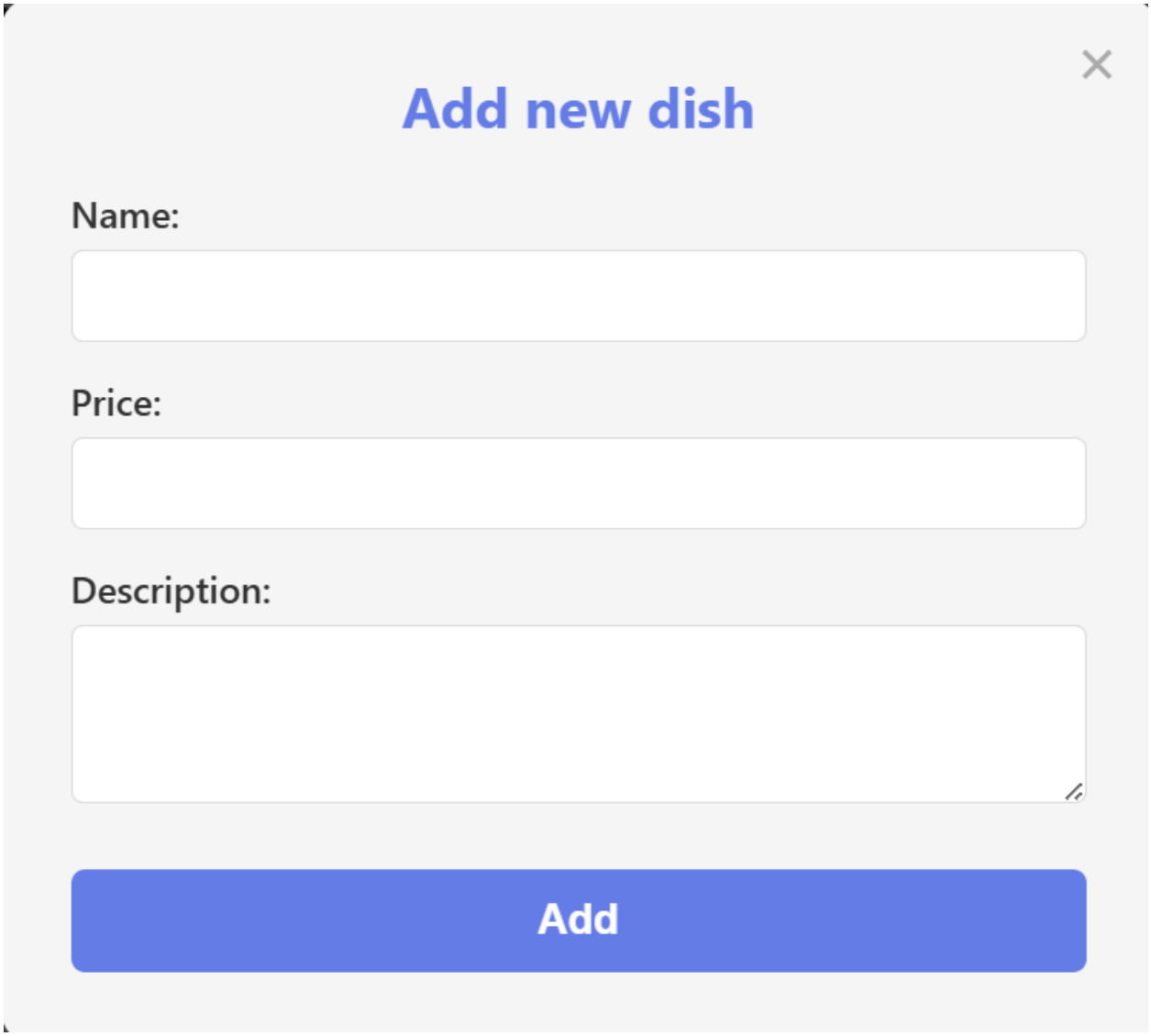
- welcome.jsp



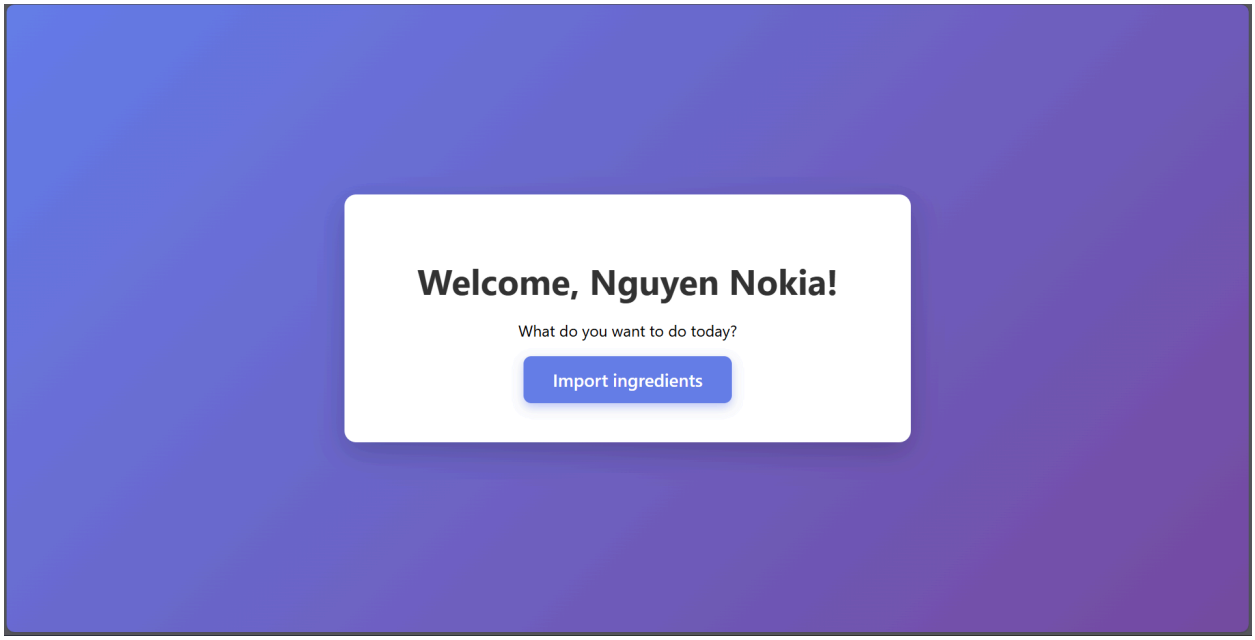
- add-dish.jsp



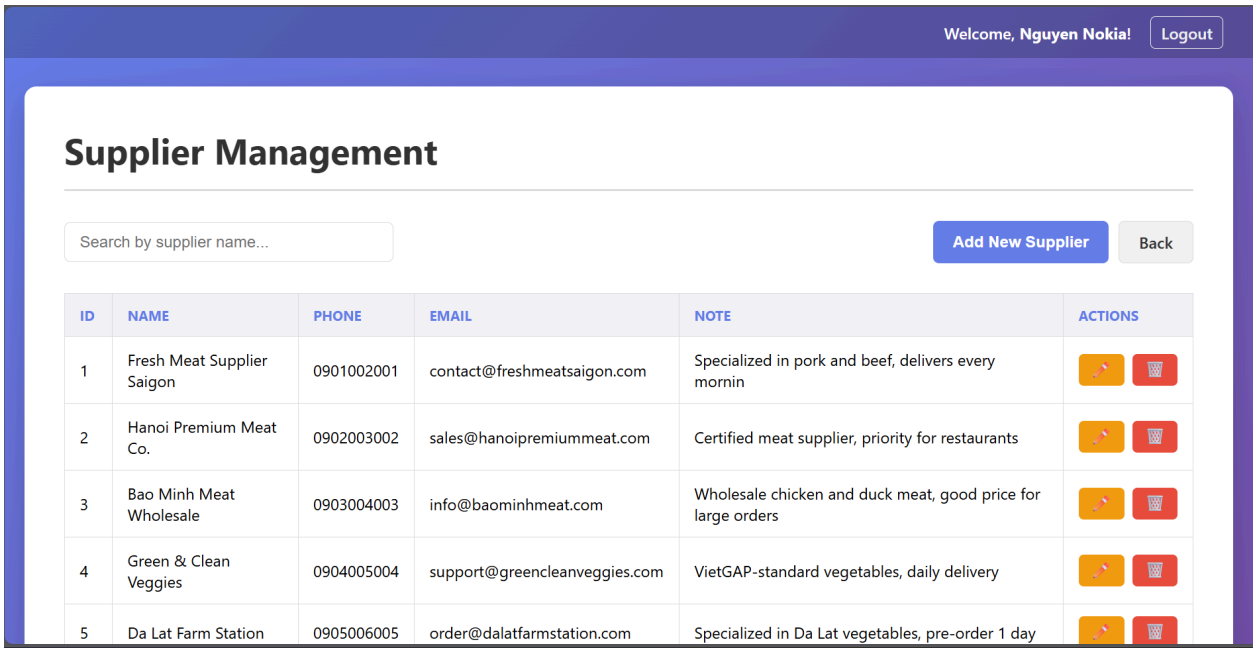
- _addDishForm.jsp



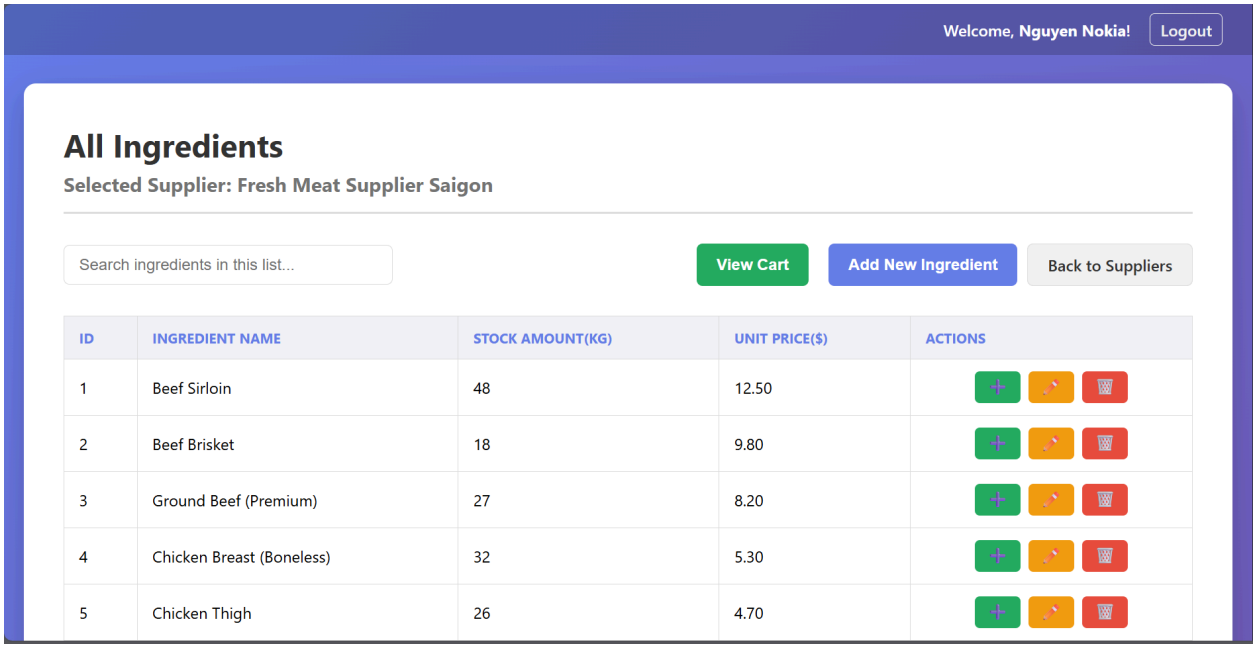
- warehouse-welcome.jsp



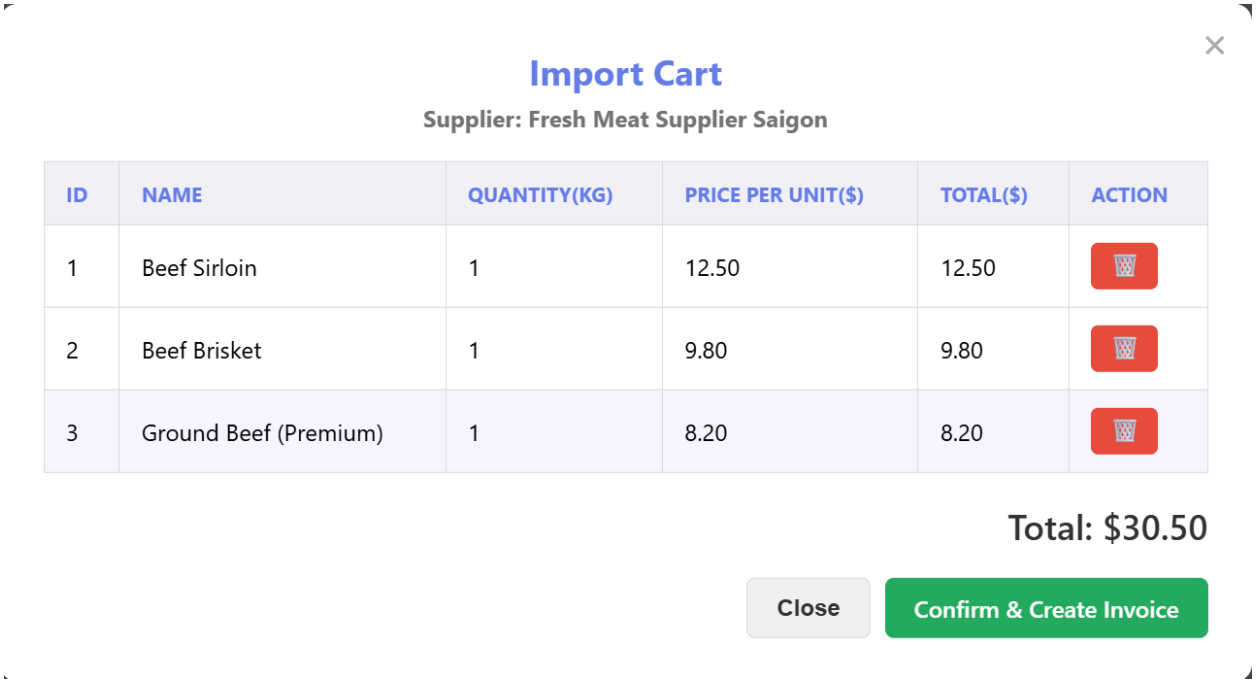
- supplier.jsp



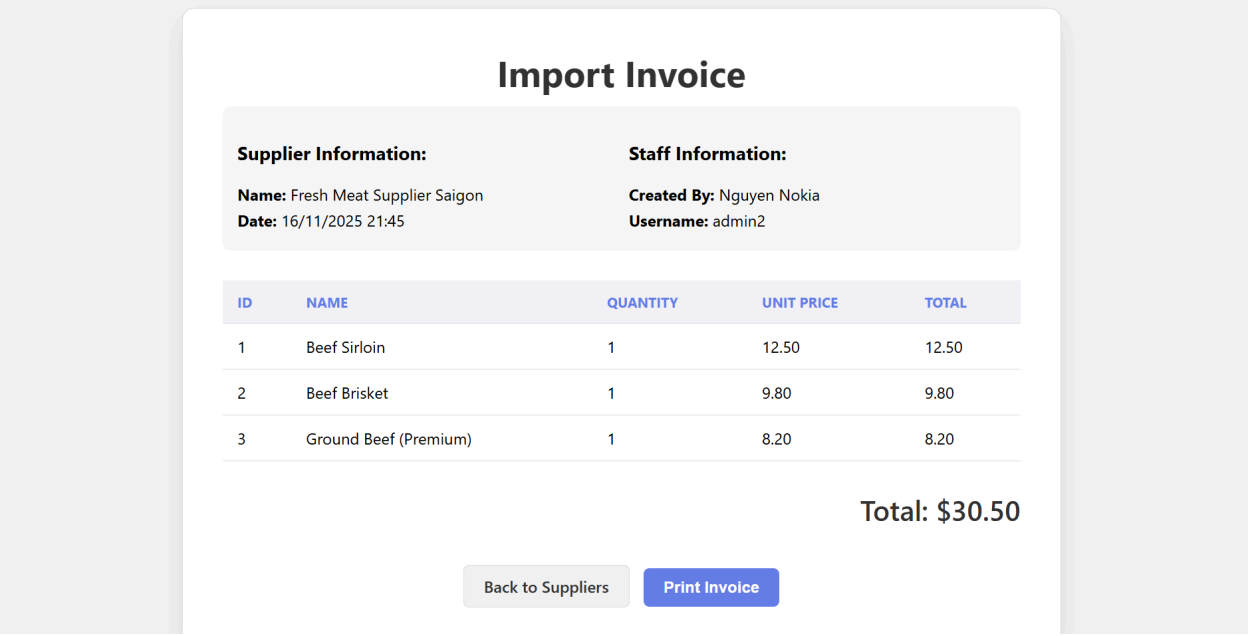
- ingredient.jsp



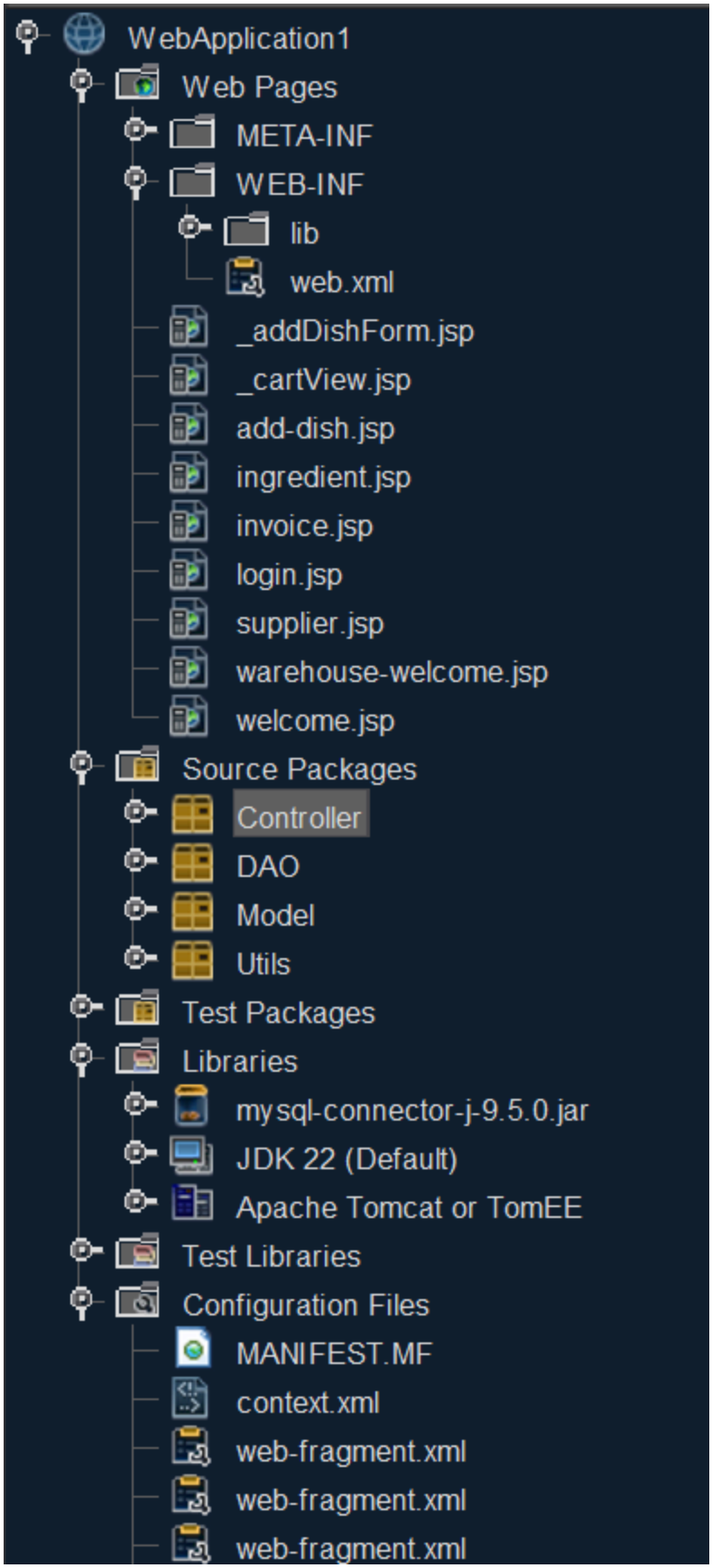
- _cartView.jsp



- invoice.jsp



2. App Structure


WebApplication1
Web Pages
META-INF
WEB-INF
lib
web.xml
_addDishForm.jsp
_cartView.jsp
add-dish.jsp
ingredient.jsp
invoice.jsp
login.jsp
supplier.jsp
warehouse-welcome.jsp
welcome.jsp
Source Packages
Controller
DAO
Model
Utils
Test Packages
Libraries
mysql-connector-j-9.5.0.jar
JDK 22 (Default)
Apache Tomcat or TomEE
Test Libraries
Configuration Files
MANIFEST.MF
context.xml
web-fragment.xml
web-fragment.xml
web-fragment.xml

3. Link source code

   1uftm3nsch/PTTK